Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

Лабораторная работы №2
«Сетевые взаимодействия»
по дисциплине «Операционные системы»

Выполнил:
студент гр. 5151003/40001
Корпусов Д. Н.

Работу принял:
Ассистент преподавателя Гавва Г. Д.

Санкт-Петербург
2026

Содержание

Введение 3-4

1. Теоретическая часть 5-6

2. Практическая часть 7-31

2.1 Сетевые взаимодействия с помощью поточных протоколов 7-21

2.2 Сетевые взаимодействия с помощью дейтаграммного протокола . 17-31

Вывод 32

Листинг А «tcpclient.cpp» 33-41

Листинг Б «tcpclient2.cpp» 42-47

Листинг В «tcpserver.cpp» 48-58

Листинг Г «udpclient.cpp» 59-64

Листинг Д «udpserver.cpp» 64-71

ВВЕДЕНИЕ

Современная разработка сетевых приложений, распределенных систем и прикладных сервисов базируется на глубоком понимании механизмов межпроцессного взаимодействия. Ключевым инструментом в этой области являются сетевые сокеты – программный интерфейс, позволяющий осуществлять обмен данными между процессами как в рамках одного хоста, так и в глобальной сети. Освоение принципов работы сокетов, особенностей транспортных протоколов TCP и UDP, а также методов организации эффективного конкурентного обслуживания клиентов является необходимым этапом подготовки разработчика сетевого ПО.

Цель работы – изучить программный интерфейс сетевых сокетов, получить навыки организации взаимодействия программ при помощи протоколов Internet и разработки прикладных сетевых сервисов.

В ходе выполнения работы планируется решить следующие основные задачи:

1. Изучить теоретические основы работы локальных (UNIX-домен) и Internet-сокетов, включая параметры их создания (пространство имен, стиль взаимодействия, протокол передачи данных).

2. Освоить системные вызовы и стандартные функции для управления соединениями: создание сокетов, привязка к адресу и порту (bind), ожидание подключений (listen, accept), а также передача и прием данных по протоколам TCP (send/recv) и UDP (sendto/recvfrom).

3. Реализовать клиент-серверные приложения для потоковой (TCP) и пакетной (UDP) передачи данных, учитывая особенности гарантированной доставки и работы с дейтаграммами.

4. Исследовать механизмы параллельного обслуживания нескольких клиентов без привлечения многопоточности, в частности, неблокирующий режим работы сокетов и функции отбора готовых дескрипторов (select, poll).

5. Разработать набор тестовых сценариев, провести верификацию работоспособности программ в типовых и граничных условиях, а также оформить результаты в соответствии с требованиями к структуре отчета.

1. Теоретическая часть

Сетевой сокет представляет собой интерфейс межпроцессного взаимодействия, идентифицируемый параметрами: домен адресации (AF_INET), тип взаимодействия (SOCK_STREAM для потоковой, SOCK_DGRAM для дейтаграммной передачи) и протокол. Создание дескриптора выполняется функцией socket(). Для серверных приложений требуется привязка сокета к локальному адресу и порту (bind()), перевод в состояние прослушивания (listen()) и приём подключений (accept()). Клиент инициирует соединение через connect(). Обмен данными осуществляется функциями send/recv (TCP) или sendto/recvfrom (UDP). TCP обеспечивает надёжную последовательную доставку за счёт механизмов управления потоком и состояния соединения, что влечёт увеличение задержки. UDP передаёт данные как отдельные пакеты без гарантий, что позволяет достигать минимальной задержки и линейной сложности обработки, но требует реализации контроля доставки и целостности на уровне приложения. По умолчанию все операции выполняются в блокирующем режиме, что приостанавливает выполнение потока до завершения сетевого вызова.

Перевод сокета в неблокирующий режим изменяет поведение системных вызовов: при отсутствии входящих данных или переполнении буфера отправки функция не приостанавливает поток, а немедленно возвращает управление с кодом ошибки EAGAIN/EWOULDBLOCK. В ОС Linux режим активируется через fcntl(fd, F_SETFL, flags | O_NONBLOCK), в Windows – через ioctlsocket(s, FIONBIO, &mode). Данный режим является обязательным условием для применения механизмов отбора событий, позволяя программе управлять состоянием множества дескрипторов в однопоточной архитектуре.

Функция select() осуществляет синхронный опрос набора файловых дескрипторов на готовность к чтению, записи или возникновению исключения. Дескрипторы передаются в виде битовых множеств типа fd_set. Перед вызовом множества должны быть очищены (FD_ZERO) и заполнены

интересующими дескрипторами (FD_SET). Первый параметр nfds задаёт значение max(fd) + 1. Структура timeval определяет время ожидания события. Возвращаемое значение указывает количество готовых дескрипторов во всех переданных множествах. Проверка состояния конкретного дескриптора выполняется макросом FD_ISSET. Ограничения механизма: жёсткий лимит на количество отслеживаемых дескрипторов (FD_SETSIZE) и необходимость пересборки множеств перед каждым вызовом.

Функция poll() (в Windows – WSAPoll()) использует массив структур pollfd, где каждый элемент содержит: дескриптор (fd), битовую маску запрашиваемых событий (events) и маску возвращаемых событий (revents). Стандартные флаги событий: POLLIN (готовность к чтению), POLLOUT (готовность к записи), POLLERR (ошибка), POLLHUP (разрыв соединения). В отличие от select, массив структур не требует пересборки перед вызовом, а верхний предел количества дескрипторов отсутствует. Функция возвращает количество элементов с ненулевым полем revents. Параметр timeout задаёт время ожидания в миллисекундах.

Сочетание неблокирующих сокетов и функций select/poll позволяет реализовать событийно-ориентированный цикл обработки соединений. Программа формирует список наблюдаемых дескрипторов (прослушивающий сокет и активные клиентские соединения), вызывает функцию отбора событий с заданным таймаутом и последовательно обрабатывает сработавшие дескрипторы: принимает новые подключения через accept(), считывает данные через recv/recvfrom или отправляет ответ через send/sendto.

2. Практическая часть

2.1 Сетевые взаимодействия с помощью поточных протоколов

Была реализована программа-сервер. Сервер построен на событийно-ориентированной модели однопоточного конкурентного обслуживания, реализуемой через системный вызов select() и неблокирующий режим работы файловых дескрипторов. В отличие от классической блокирующей реализации, где каждый клиент обслуживается в отдельном потоке или процессе, данное решение позволяет одному потоку управления одновременно отслеживать состояние произвольного множества открытых соединений, не тратя ресурсы на переключение контекста и синхронизацию общих данных.

Алгоритм работы сервера представляет собой непрерывный цикл обработки событий, состоящий из четырёх логических фаз:

1. Подготовка множеств дескрипторов. Перед каждым вызовом select() происходит очистка структур fd_set (read_set, write_set). В множество чтений добавляется дескриптор прослушивающего сокета (listen_fd) и дескрипторы всех активных клиентских соединений. В множество записей добавляются только те клиентские дескрипторы, у которых в выходном буфере (out_buf) присутствует неотправленная очередь данных. Параллельно вычисляется значение max_fd как максимальный номер дескриптора среди переданных, что требуется спецификацией select().

2. Ожидание сетевых событий. Вызов select() блокирует выполнение потока до момента, пока хотя бы один из переданных дескрипторов не перейдёт в состояние готовности (поступление данных, возможность отправки, ошибка или новое подключение), либо не возникнет системная ошибка. Таймаут не устанавливается, что обеспечивает минимальную задержку реакции на сетевую активность.

3. Обработка новых подключений. Если прослушивающий сокет отмечен как готовый к чтению, сервер в цикле вызывает accept() до тех пор, пока не возникнет ошибка EAGAIN/EWOULDBLOCK, что сигнализирует об

исчерпании очереди ожидающих соединений. Каждый принятый сокет немедленно переводится в неблокирующий режим, инициализируется структура ClientState и размещается в первом свободном слоте статического массива клиентов.

4. Обработка данных и состояния клиентов. Сервер последовательно проходит по всем активным слотам: если дескриптор присутствует в read_set, вызывается функция чтения. Данные передаются во входной динамический буфер. На этапе инициализации чтения ожидается 3-байтовая команда (put или get). После её получения определяется режим работы. В режиме PUT запускается парсер бинарных сообщений: извлекаются индекс, телефоны, время и текст, производится валидация, данные записываются в файл msg.txt, а в выходной буфер клиента формируется подтверждение ok. При получении текстового поля stop выставляется флаг завершения сервера. В режиме GET сервер считывает историю из msg.txt, формирует поток бинарных пакетов в выходной буфер и устанавливает флаг закрытия соединения после полной отправки данных. Если дескриптор присутствует в write_set, сервер пытается отправить содержимое out_buf. При частичной отправке (характерной для TCP-потоков) отслеживается количество реально переданных байт (out_sent). Если буфер опустел и установлен флаг close_after_send, соединение закрывается, слот освобождается.

При получении recv == 0 (корректное закрытие клиентом) или сетевой ошибке дескриптор закрывается, буферы освобождаются, состояние слота сбрасывается.

Цикл прерывается исключительно по команде stop, полученной от клиента в режиме PUT. При этом сервер гарантирует отправку финального подтверждения ok перед завершением работы, что предотвращает потерю данных в буферах сетевой подсистемы. Блок-схема алгоритма представлена на рисунке 1.

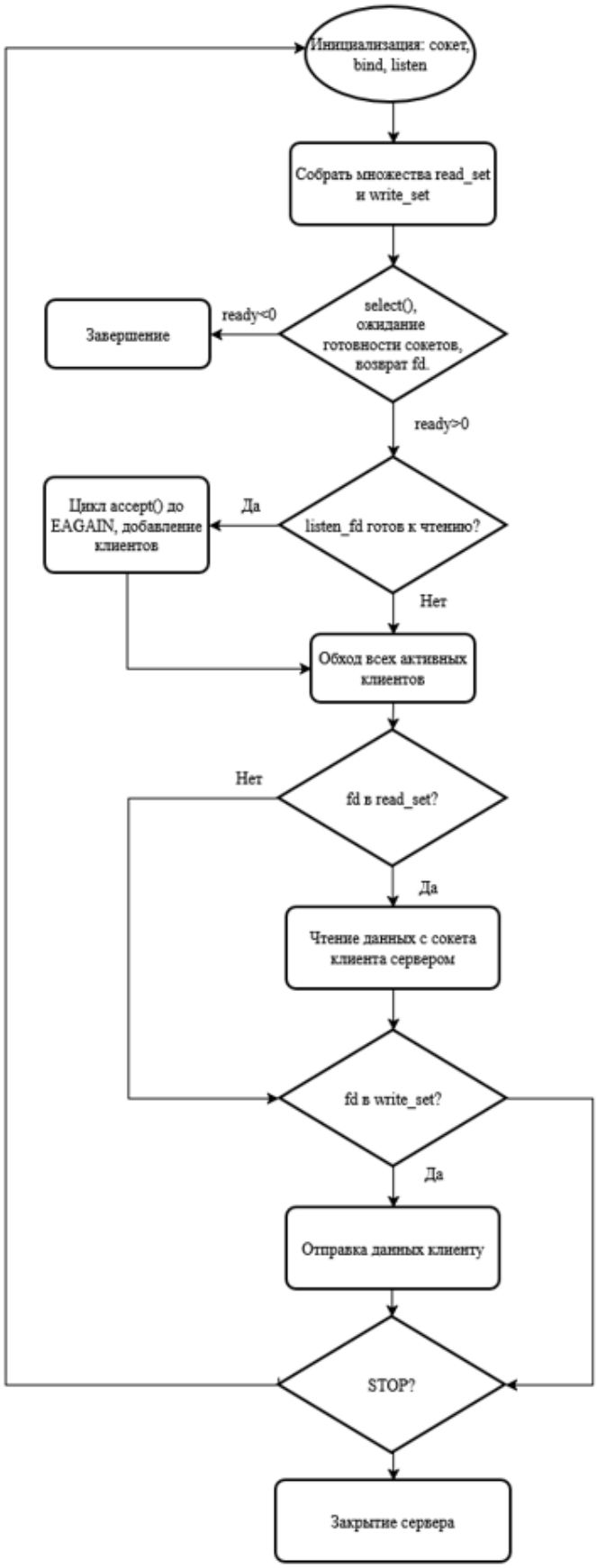


Рисунок 1 – Блок-схема алгоритма работы tcp-сервера

В сервере используется одна основная структура ClientState, описывающая полное состояние активной TCP-сессии. Динамические буферы обусловлены потоковой природой TCP: сегменты могут фрагментироваться, объединяться или доставляться с задержкой, поэтому сервер не может полагаться на целые сообщения в одном вызове recv().

Таблица 1 – Поля структуры ClientState

Поле	Тип	Назначение
fd	int	Файловый дескриптор сокета клиента. Значение -1 – маркер свободного слота в статическом массиве clients.
addr	struct sockaddr_in	IP-адрес и порт подключившегося узла. Используется для формирования префикса IP:PORT при записи в msg.txt.
mode	int (enum ClientMode)	Текущее состояние протокола. Определяет логику парсинга входного буфера и формирование ответов.
close_after_send	int(флаг)	Принудительно закрывает соединение после полной считывания выходного буфера. Активируется в режиме GET после формирования всего ответа.
stop_after_send	int(флаг)	Фиксирует получение текстового поля stop в режиме PUT. Позволяет серверу завершить основной цикл только после гарантированной доставки подтверждения ok.

Продолжение таблицы 1

Поле	Тип	Назначение
peer_closed	int(флаг)	Индикатор корректного или аварийного разрыва связи (recv==0 или сетевая ошибка). Прекращает попытки чтения и инициирует очистку слота.
in_buf	char*	Динамический входной буфер накопления TCP-потока. in_buf – всё, что пришло от клиента.
out_buf	size_t	Динамический выходной буфер-очередь. out_buf – всё, что сервер хочет отправить клиенту.
out_sent	size_t	Показывает, сколько из out_buf удалось отправить.

Таблица 2 – Реализованные функции tcp-сервера

Название функции	Назначение
main	Главная функция сервера: создаёт сокет, настраивает его, запускает цикл select(), принимает клиентов, читает/отправляет данные, завершает сервер
CheckPort	Проверяет аргумент порта из командной строки и переводит строку(порт) в число
SetNonBlocking	Переводит сокет в неблокирующий режим
InitClient	Сбрасывает структуру клиента в начальное пустое состояние
CloseClient	Закрывает сокет клиента и очищает его структуру
FlushOutput	Пытается отправить клиенту всё, что лежит в out_buf
QueueOk	Добавляет ответ ok в выходной буфер клиента
ValidText	Проверяет, что текст сообщения допустимый

Продолжение таблицы 2

Название функции	Назначение
WriteMessageToFile	Записывает одно принятое сообщение в msg.txt
QueueGetMessage	Упаковывает одну запись в бинарный формат для ответа на get
LoadHistoryForGet	Читает msg.txt, разбирает строки и складывает всю историю в out_buf клиента

Клиентская программа tcpclient.cpp предназначена для считывания сообщений из текстового файла и их последовательной передачи удалённому серверу по TCP-протоколу в формате, заданном условием лабораторной работы. Адрес сервера и имя входного файла передаются программе через аргументы командной строки.

Структура кода программы включает несколько основных частей. В начале файла объявляется структура Message, описание которой представлено в таблице 3, предназначенная для хранения одного сообщения в разобранном виде.

Таблица 3 – Поля структуры Message

Поле	Тип	Назначение
index	unsigned int	Номер сообщения во входном файле. Используется при формировании первых 4 байт сообщения по протоколу
phone1	char[13]	Первый номер телефона. Хранится как строка длиной 12 символов с дополнительным '\0' в конце
phone2	char[13]	Второй номер телефона. Аналогично хранится как строка из 12 символов с завершающим нулевым байтом

Продолжение таблицы 3

hour	unsigned char	Значение часов из поля времени сообщения
minute	unsigned char	Значение минут из поля времени сообщения
second	unsigned char	Значение секунд из поля времени сообщения

Названия функций и их назначение приведены в таблице 4.

Таблица 4 – Функции клиентской программы tcpclient.cpp

Название функции	Назначение
main	Основная функция программы: обрабатывает аргументы командной строки, устанавливает соединение, считывает сообщения из файла, передаёт их серверу и получает подтверждения
CheckPhone	Проверяет корректность формата телефонного номера +7xxxxxxxxxx
CheckTime	Проверяет допустимость значений часов, минут и секунд
CheckText	Проверяет корректность текстовой части сообщения
SendAll	Выполняет полную отправку всех байтов сообщения в сокет, даже если один вызов send() передал только часть данных
RecvAll	Выполняет приём заданного количества байтов из сокета, например двухбайтового подтверждения ok
SendMessage	Формирует одно полное сообщение по протоколу и отправляет его серверу
ReadLongLine	Считывает одну строку входного файла целиком, в том числе при большой длине сообщения
Finish	Завершает работу программы: закрывает сокет, входной файл и освобождает ресурсы WinSock

Алгоритм работы программы можно описать следующим образом. После запуска программа проверяет количество аргументов командной строки, открывает входной файл и инициализирует библиотеку WinSock. Затем из строки вида IP:порт выделяются адрес сервера и номер порта, формируется структура сетевого адреса и выполняется подключение к серверу. В соответствии с условием задания при неудачном подключении программа выполняет до десяти попыток соединения с интервалом 100 мс.

После успешного подключения клиент передаёт серверу трёхбайтовую команду put, которая сообщает о начале передачи сообщений. Далее программа последовательно считывает строки из входного файла, пропуская пустые и некорректные строки. Для каждой корректной строки выделяются два номера телефона, время и текст сообщения. После этого формируется бинарное сообщение в требуемом формате: четырёхбайтовый номер сообщения в сетевом порядке байтов, два телефонных номера по 12 байт, три байта времени, текст сообщения и завершающий нулевой байт. Подготовленное сообщение передаётся серверу по TCP-соединению.

После отправки каждого сообщения клиент ожидает от сервера подтверждение ok. Только после получения подтверждения программа переходит к передаче следующего сообщения. Это гарантирует, что клиент завершает работу только после получения подтверждения по последнему сообщению, как того требует условие задания. По окончании передачи всех сообщений программа закрывает соединение и завершает работу.

Для наглядного представления порядка обмена данными между клиентом и сервером далее была составлена временная диаграмма взаимодействия, представленная на рисунке 2.

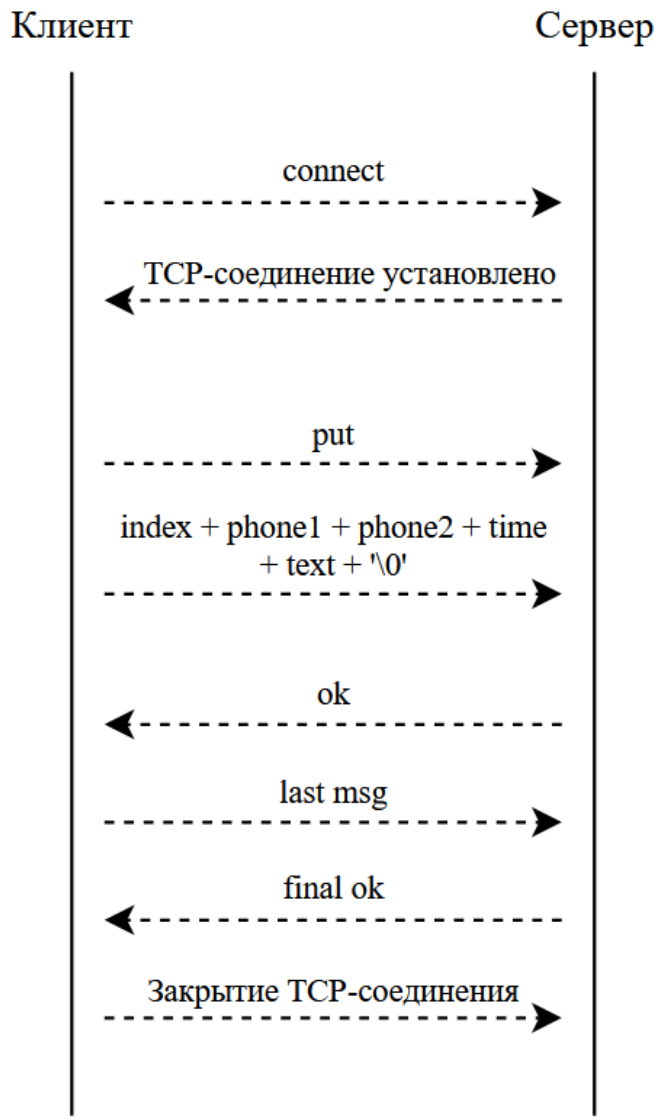


Рисунок 2 – Временная диаграмма взаимодействия клиента и сервера

Из представленной диаграммы видно, что клиент устанавливает соединение с сервером один раз в начале работы, после чего использует его для последовательной передачи всех сообщений из входного файла и завершает работу только после получения подтверждения по последнему сообщению.

Проверка корректности работы клиентской программы tcpclient.cpp на начальном этапе выполнялась с использованием эмулятора сервера tcpserveremul.rb. В ходе проверки контролировались корректность установки TCP-соединения, передача служебной команды put, последовательная отправка сообщений из входного файла в требуемом бинарном формате, а

также получение подтверждений ok от сервера на каждое переданное сообщение.

В качестве первого теста была выполнена проверка работы клиентской программы tcpclient.cpp с использованием эмулятора сервера tcpserveremul.rb. Входной файл содержал одно сообщение со служебным словом stop.

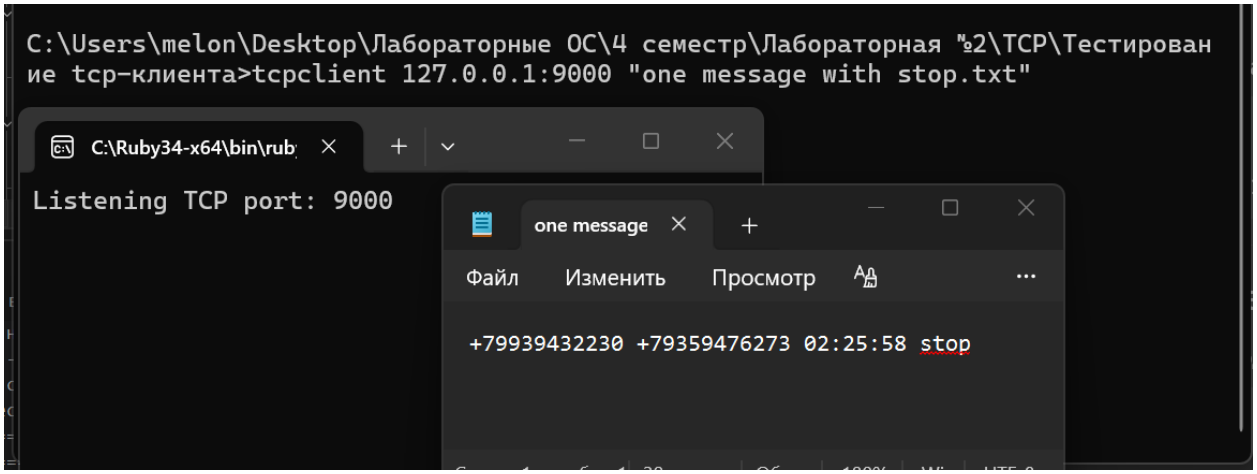


Рисунок 3 – Проверка работы tcpclient.cpp с использованием эмулятора tcpserveremul.rb

На рисунке 3 представлен пример запуска клиентской программы tcpclient.cpp при тестировании с эмулятором сервера tcpserveremul.rb. Входной файл содержит одно сообщение со служебным словом stop, а клиент подключается к локальному серверу по адресу 127.0.0.1:9000.

В результате выполнения первого теста клиентская программа успешно установила TCP-соединение с сервером и передала сообщение из входного файла в соответствии с заданным протоколом. Эмулятор сервера корректно принял сообщение, сохранил его в файл msg.txt в текстовом виде, включая адрес клиента, номера телефонов, время и текст сообщения stop, что представлено на рисунке 4, после чего завершил работу, как и предусмотрено условием задания.

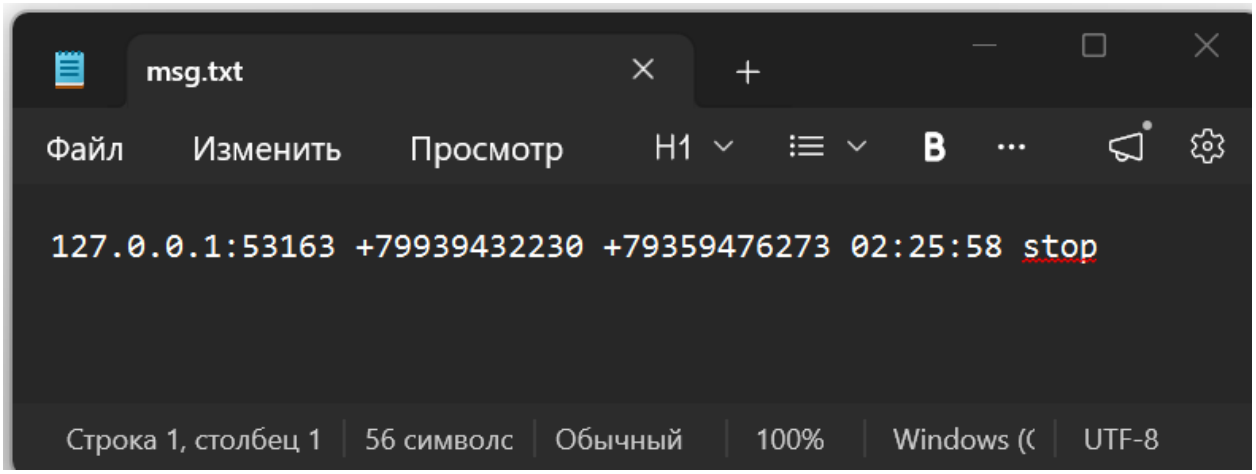


Рисунок 4 – Результат обработки сообщения сервером при тестировании клиентской программы

В качестве следующего теста была выполнена проверка корректности передачи нескольких сообщений большой длины. Для этого использовался входной файл «many very large messages.txt», представленный на рисунке 5, содержащий набор сообщений с существенно увеличенным текстовым полем, а также завершающее служебное сообщение stop.

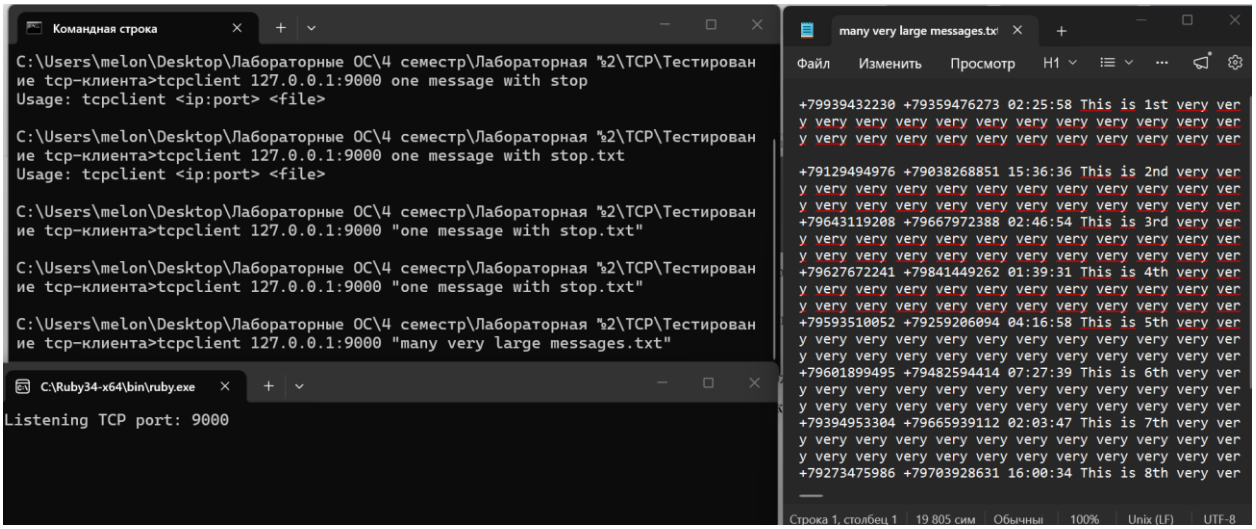


Рисунок 5 – Подготовка и запуск теста с несколькими длинными сообщениями

Результат обработки сообщения представлен на рисунке 6.

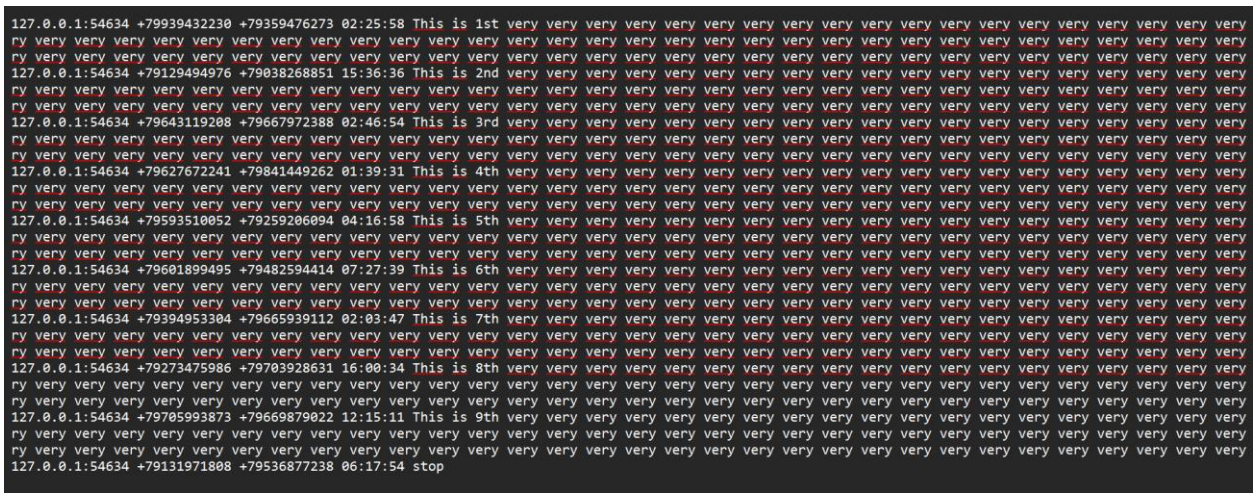

127.0.0.1:54634 +79939432230 +79359476273 02:25:58 This is 1st very very very



Рисунок 8 – Результат обработки сообщения сервером

Возможность передачи сообщений большой длины обеспечивается особенностями реализации клиентской программы. Чтение строк из входного файла выполняется функцией ReadLongLine, которая позволяет считывать одну строку целиком, даже если её длина превышает размер обычного фиксированного буфера. Строка читается частями и по мере чтения динамически наращивается в памяти с помощью realloc. После этого клиент формирует полное бинарное сообщение и передаёт его серверу. Непосредственная отправка данных выполняется функцией SendAll, которая гарантирует передачу всех байтов сообщения по TCP-соединению, даже если один вызов send() передал только часть данных.

Дополнительная программа tcpclient2.cpp предназначена для получения от сервера всех ранее сохранённых сообщений по специальной команде get. Клиент2 запускается с аргументами командной строки в виде IP:Port get FILENAME, подключается к серверу по TCP, отправляет трёхбайтовую команду get, после чего принимает от сервера последовательность сообщений в бинарном формате, заданном протоколом. Каждое принятое сообщение

сохраняется в выходной файл FILENAME в текстовом виде, при этом перед сообщением добавляются IP-адрес и TCP-порт сервера.

После запуска программа проверяет корректность аргументов командной строки. Второй аргумент должен содержать строку get, а первый аргумент должен задавать адрес сервера в формате IP:Port. Далее функция parse_server_address выделяет из этой строки IP-адрес и номер порта, заполняет структуру sockaddr_in, а также формирует строковый префикс вида IP:Port, который затем будет добавляться в начало каждой строки выходного файла.

На следующем этапе программа создаёт выходной файл, инициализирует библиотеку WinSock, создаёт TCP-сокет и настраивает таймауты операций чтения и записи. Подключение к серверу выполняется функцией connect_with_retry, которая реализует до десяти попыток соединения с интервалом 100 мс между ними. После успешного подключения клиент отправляет серверу команду get тремя байтами.

После отправки команды начинается приём сообщений от сервера. Каждое сообщение принимается функцией recv_message. Сначала клиент считывает 4 байта служебного номера сообщения, затем два телефонных номера по 12 байт, далее три байта времени и после этого посимвольно принимает текст сообщения до тех пор, пока не встретится завершающий нулевой байт '\0'. Номер сообщения носит служебный характер, поэтому в выходной файл он не записывается.

После полного приёма очередного сообщения функция write_message сохраняет его в файл в текстовом формате. В начале строки записывается строковый префикс с IP-адресом и портом сервера, затем первый и второй номера телефонов, время в формате hh:mm:ss и текст сообщения. Когда сервер завершает передачу и закрывает соединение, функция recv_message перестаёт получать данные, цикл приёма завершается, после чего программа закрывает файл, сокет и завершает работу.

Таким образом, алгоритм работы tcpclient2.cpp состоит из разбора аргументов, подключения к серверу, отправки команды get, последовательного приёма всех сохранённых сервером сообщений и их записи в выходной файл в требуемом текстовом формате.

В качестве теста для программы tcpclient2.cpp была выполнена проверка получения ранее сохранённых сервером сообщений по команде get. Для этого tcpclient2 подключался к серверу, отправлял команду get и сохранял принятые сообщения в указанный выходной файл, как показано на рисунке

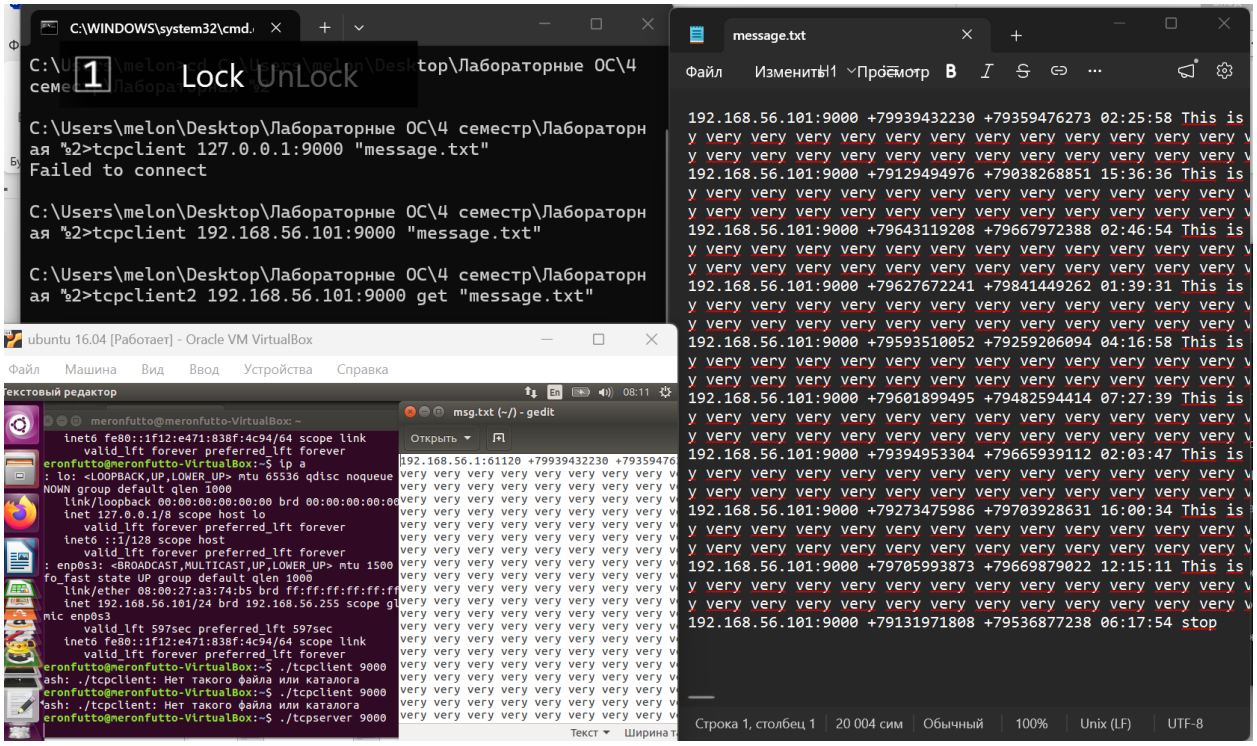


Рисунок 9 – Результат обработки команды get сервером

Результат тестирования показал, что tcpclient2 успешно получил сообщения от сервера и корректно сохранил их в выходной файл message.txt. Каждая строка выходного файла предваряется IP-адресом и портом сервера

2.2 Сетевые взаимодействия с помощью дейтаграммного протокола UDP

Параллельное обслуживание клиентов в программе udpserver.cpp реализовано в рамках однопоточной модели с использованием механизма WSAPoll. Сервер создаёт набор UDP-сокетов для прослушивания заданного диапазона портов и далее в основном цикле ожидает возникновения событий

чтения на любом из них. Такой подход позволяет одной программе без создания отдельных потоков последовательно обрабатывать сообщения, поступающие от нескольких клиентов на разные сокеты.

Каждый клиент в UDP-сервере идентифицируется по паре IP-адрес: порт отправителя. При получении очередной дейтаграммы сервер определяет, существует ли уже запись о данном клиенте во внутреннем списке. Если клиент обращается впервые, для него создаётся новая структура с данными о времени последней активности и списком ранее принятых номеров сообщений. Если клиент уже известен серверу, обновляется его время активности и выполняется проверка, не является ли полученное сообщение дубликатом.

После приёма дейтаграммы сервер извлекает из неё номер сообщения, служебные поля и текстовую часть. Если номер сообщения от данного клиента ещё не встречался, сообщение считается новым, сохраняется в файл msg.txt, а его номер добавляется в список принятых сообщений соответствующего клиента. Если сообщение уже было принято ранее, повторная запись в файл не выполняется, однако сервер всё равно формирует ответ клиенту согласно протоколу.

Ответ сервера представляет собой набор номеров сообщений, успешно принятых от данного клиента. Для этого сервер формирует буфер ответа из последних идентификаторов сообщений клиента, переводя каждый номер в сетевой порядок байтов. После этого ответ отправляется обратно по адресу отправителя с использованием функции sendto.

Дополнительно в основном цикле сервера выполняется обслуживание служебных ситуаций. Если текст полученного сообщения равен stop, сервер после отправки ответа завершает работу. Также после каждой итерации цикла выполняется очистка списка клиентов: записи, для которых время последней активности превышает 30 секунд, удаляются из внутренней структуры данных.

Таким образом, параллельное обслуживание нескольких клиентов в UDP-сервере достигается без использования многопоточности. Сервер последовательно обрабатывает события от разных клиентов, используя механизм опроса сокетов WSAPoll, внутренний список клиентов и неблокирующий режим работы сокетов.

Для наглядного представления последовательности обработки дейтаграмм и обслуживания нескольких клиентов была составлена блок-схема алгоритма работы UDP-сервера.



Рисунок 10 – Блок-схема алгоритма параллельного обслуживания клиентов в UDP-сервере

Для хранения информации о клиентах и принятых сообщениях в программе udpserver.cpp используются структура данных Client, поля которой представлены в таблице 5, позволяющая идентифицировать клиентов, учитывать время их последней активности и хранить список уже полученных идентификаторов сообщений.

Таблица 5 – Структура данных Client в udpserver.cpp

Поле	Тип данных	Назначение
ip	uint32_t	IP-адрес клиента
port	uint16_t	Порт клиента
last_activity	time_t	Время последней активности клиента, используется для удаления неактивных записей
received_ids	std::vector<uint32_t>	Список идентификаторов сообщений, уже полученных от данного клиента

Названия реализованных функций и их назначение приведены в таблице 6.

Таблица 6 – Функции программы udpserver.cpp

Функция	Назначение
CheckPort	Проверяет корректность номеров начального и конечного портов, переданных в аргументах командной строки
CreateSockets	Создаёт UDP-сокеты для заданного диапазона портов, переводит их в неблокирующий режим и выполняет привязку к адресам
CreateClient	Формирует новую запись клиента с IP-адресом, портом и временем последней активности
main	Основная функция программы: выполняет инициализацию WinSock, создание сокетов, ожидание событий через WSAPoll, приём и обработку дейтаграмм, отправку ответов и завершение работы сервера

Клиентская программа udpclient.cpp предназначена для считывания сообщений из текстового файла и их передачи удалённому серверу по UDP-протоколу. Адрес сервера и имя входного файла передаются программе через аргументы командной строки при запуске. В отличие от TCP-клиента,

взаимодействие с сервером осуществляется без установления постоянного соединения, а обмен данными выполняется с помощью UDP-дейтаграмм.

Для хранения одного сообщения, а также подготовленных к отправке UDP-дейтаграмм в программе udpclient.cpp используются структуры данных Message и PreparedMessage. Их поля и назначение приведены в таблице 7, 8 соответственно.

Таблица 7 – Структура данных Message в udpclient.cpp

Поле	Тип данных	Назначение
index	uint32_t	Номер сообщения во входном файле, используемый в протоколе
phone1	char[13]	Первый номер телефона в текстовом виде;
phone2	char[13]	Второй номер телефона в текстовом виде
hour	uint8_t	Значение часов
minute	uint8_t	Значение минут
second	uint8_t	Значение секунд

Таблица 8 – Структура данных PreparedMessage в udpclient.cpp

Поле	Тип данных	Назначение
index	uint32_t	Номер сообщения, соответствующий данной подготовленной дейтаграмме
datagram	unsigned char*	Указатель на буфер, содержащий готовую UDP-дейтаграмму в бинарном формате
datagram_len	size_t	Длина подготовленной дейтаграммы в байтах

Названия функций и их назначение приведены в таблице 9.

Таблица 9 – Функции программы udpclient.cpp

Функция	Назначение
BuildDatagram	Формирует готовую UDP-дейтаграмму в бинарном формате по данным структуры Message и текстовой части сообщения

ParseServerAddress	Разбирает строку вида IP:порт, проверяет корректность адреса и заполняет структуру sockaddr_in
main	Основная функция программы: проверяет аргументы, создаёт UDP-сокет, считывает сообщения из файла, подготавливает дейтаграммы, отправляет их серверу, принимает подтверждения и завершает работу клиента

На начальном этапе работы программа проверяет количество аргументов командной строки, разбирает адрес сервера и создаёт UDP-сокет. Затем открывается входной файл, из которого построчно считываются сообщения. При чтении из строки удаляются символы перевода строки, пустые строки пропускаются. Для каждой корректной строки выделяются два номера телефона, время и текст сообщения. После этого заполняется структура Message, а затем вызывается функция BuildDatagram, которая создаёт готовую бинарную дейтаграмму.

Каждая подготовленная дейтаграмма сохраняется в динамическом массиве msgs как элемент структуры PreparedMessage. Таким образом, клиент заранее формирует набор всех сообщений, которые затем будут отправляться серверу. Для каждого сообщения сохраняются его номер, указатель на буфер дейтаграммы и её длина.

После подготовки всех сообщений программа переходит к этапу передачи данных. Сначала клиент однократно отправляет серверу все подготовленные дейтаграммы. Затем начинается основной цикл ожидания подтверждений. Для контроля поступления ответов используется функция select с таймаутом 100 мс. Если в течение этого времени ответ от сервера не поступил, клиент повторно отправляет все сообщения, для которых подтверждение ещё не было получено. Такой механизм позволяет компенсировать возможные потери UDP-дейтаграмм.

При поступлении ответа от сервера клиент вызывает recvfrom и получает буфер подтверждения. Ответ сервера содержит последовательность номеров сообщений, каждое из которых занимает 4 байта. Клиент поочерёдно извлекает эти номера, переводит их из сетевого порядка байтов в локальный и отмечает соответствующие сообщения как подтверждённые в массиве acked. Одновременно увеличивается счётчик подтверждённых сообщений acked_count.

Работа клиента продолжается до тех пор, пока количество подтверждённых сообщений не достигнет значения target. Значение target выбирается как количество подготовленных сообщений, если их меньше 20, либо 20, если сообщений больше. После достижения этого условия клиент освобождает динамически выделенную память, закрывает входной файл и UDP-сокет, после чего завершает работу.

Таким образом, алгоритм работы udpclient.cpp основан на предварительном формировании всех UDP-дейтаграмм, их отправке серверу, ожидании подтверждений и повторной передаче неподтверждённых сообщений. Такой подход позволяет реализовать обмен данными поверх ненадёжного дейтаграммного протокола UDP и обеспечить доставку сообщений за счёт повторных отправок.

Для более наглядного представления процесса обмена данными между клиентом и сервером по протоколу UDP далее была составлена временная диаграмма взаимодействия, представленная на рисунке

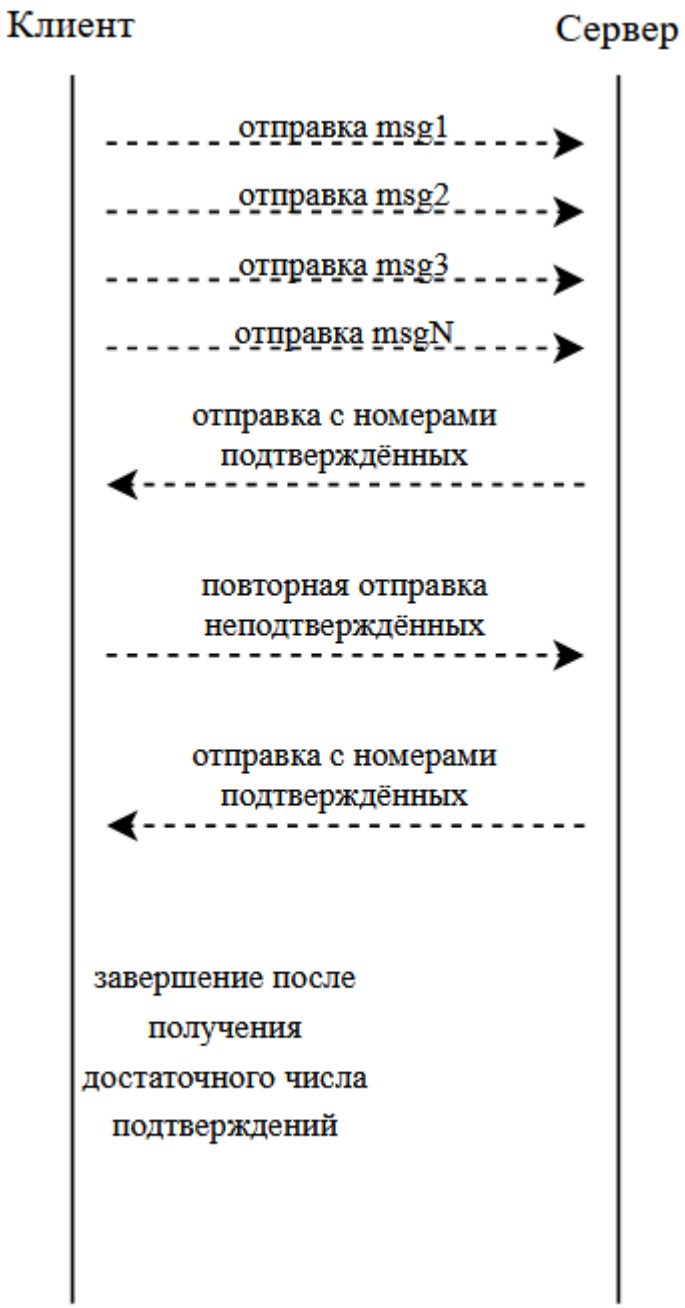


Рисунок 11 – Временная диаграмма взаимодействия клиента и сервера

Из представленной диаграммы видно, что в отличие от TCP-взаимодействия обмен выполняется без установления соединения. Клиент отправляет серверу дейтаграммы с сообщениями, получает ответы с подтверждёнными идентификаторами и при необходимости повторно передаёт сообщения, подтверждение по которым ещё не было получено.

Для проверки корректности работы программ udpclient.cpp и udpserver.cpp было проведено тестирование.

В качестве первого теста была выполнена проверка передачи одного сообщения, содержащего служебное слово stop. Для этого использовался входной файл, состоящий из одной строки с корректно заданными номерами телефонов, временем и текстом сообщения stop, как представлено на рисунке 12.

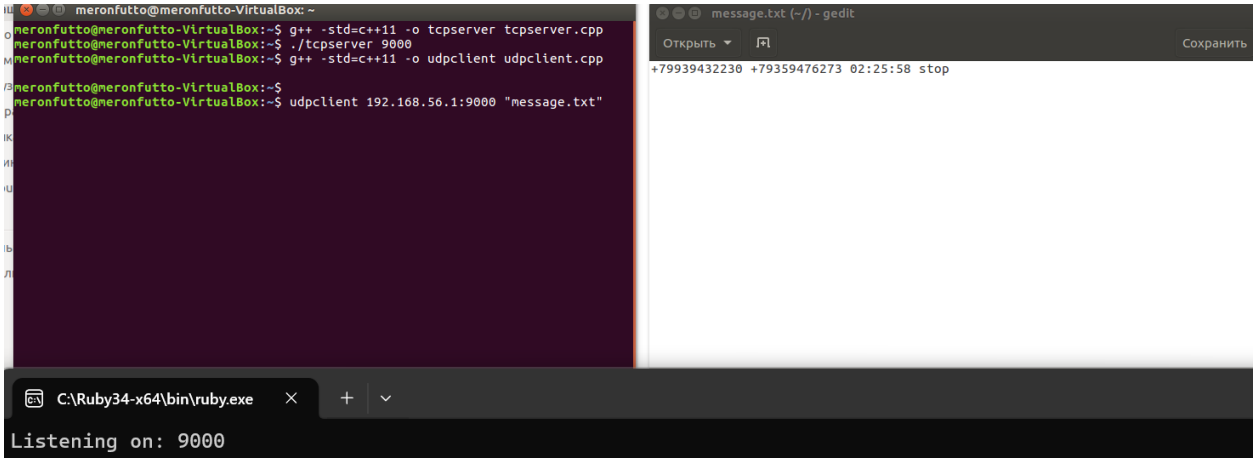


Рисунок 12 – Проверка передачи одного сообщения программой udpclient.cpp

Результат представлен на рисунке 13.

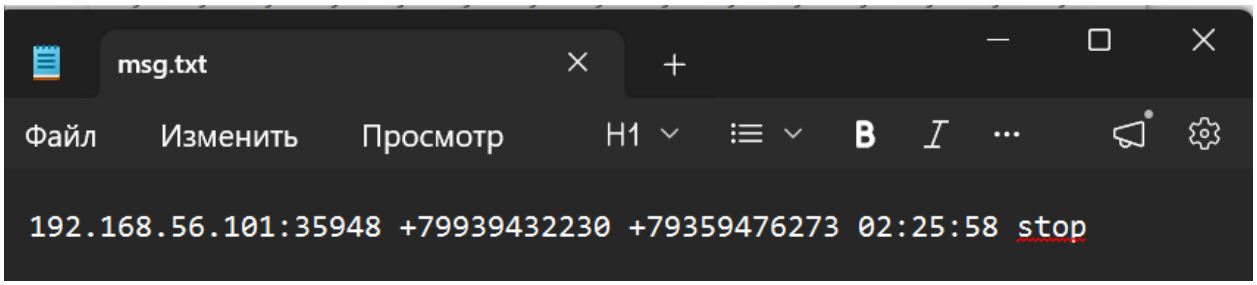


Рисунок 13 – Результат обработки сервером сообщения

Далее была проведена проверка работы UDP-программ в случае одновременного обращения нескольких клиентов к серверу. Для тестирования использовались два входных файла: один с набором обычных сообщений, другой — с сообщением stop, предназначенным для завершения работы сервера.

В результате выполнения теста было установлено, что UDP-сервер корректно обработал сообщения, поступившие от нескольких клиентов. Сообщения из первого входного файла были успешно приняты сервером и записаны в файл msg.txt с указанием IP-адреса и порта клиента. После этого

сервер также обработал сообщение со служебным словом stop, поступившее от второго клиента, и завершил работу в соответствии с требованиями задания.

ВЫВОД

В результате выполнения лабораторной работы были реализованы программы обмена сообщениями по протоколам TCP и UDP. Разработанные клиенты и серверы успешно передают, принимают и сохраняют сообщения, а также корректно обрабатывают служебные команды. Тестирование подтвердило правильность работы программ и соответствие полученных результатов условиям задания.

Приложение А «tcpclient.cpp»

```
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#pragma comment(lib, "ws2_32.lib")

struct Message {
    unsigned int index;         // Номер сообщения в файле: 0, 1,
2, ...
    char phone1[13];             // Первый телефон. 13, а не 12,
чтобы добавить '\0' в конец.
    char phone2[13];            // Второй телефон.
    unsigned char hour;         // Часы 0..23
    unsigned char minute;       // Минуты 0..59
    unsigned char second;       // Секунды 0..59
};

// Проверяем, что телефон имеет вид +7xxxxxxxxxx.
int CheckPhone(const char *phone) {
    int i;

    if (phone == NULL) return 0;
    if (strlen(phone) != 12) return 0;
    if (phone[0] != '+' || phone[1] != '7') return 0;

    for (i = 2; i < 12; ++i) {
        if (phone[i] < '0' || phone[i] > '9') return 0;
    }

    return 1;
}

// Проверяем корректность времени.
int CheckTime(int hh, int mm, int ss) {
    if (hh < 0 || hh > 23) return 0;
    if (mm < 0 || mm > 59) return 0;
    if (ss < 0 || ss > 59) return 0;
    return 1;
}

// В тексте не должно быть пустой строки и переводов строки.
int CheckText(const char *text) {
    int i;

    if (text == NULL || text[0] == '\0') return 0;

    for (i = 0; text[i] != '\0'; ++i) {
```

```
        if (text[i] == '\n' || text[i] == '\r') return 0;
    }

    return 1;
}

// send может отправить не все байты сразу, поэтому досылаем в
цикле.
int SendAll(SOCKET s, const char *data, int size) {
    int total = 0; // Сколько байт уже удалось отправить.

    while (total < size) {
        int sent = send(s, data + total, size - total, 0);
        if (sent <= 0) {
            printf("send failed: %d\n", WSAGetLastError());
            return 0;
        }
        total += sent;
    }

    return 1;
}

// recv тоже может вернуть только часть данных, поэтому дочитываем
до конца.
int RecvAll(SOCKET s, char *data, int size) {
    int total = 0; // Сколько байт уже получили.

    while (total < size) {
        int received = recv(s, data + total, size - total, 0);
        if (received == SOCKET_ERROR) {
            printf("recv failed: %d\n", WSAGetLastError());
            return 0;
        }
        if (received == 0) {
            printf("Connection closed by server\n");
            return 0;
        }
        total += received;
    }

    return 1;
}

// Собираем одно полное сообщение по протоколу и отправляем его
целиком.
int SendMessage(SOCKET s, const Message *msg, const char *text) {
    size_t text_len = strlen(text);
    size_t msg_size = 4 + 12 + 12 + 3 + text_len + 1;
    char *sendbuf = (char *)malloc(msg_size);
    unsigned int net_index;
    size_t pos = 0;
    int result = 0;
```

```
    if (sendbuf == NULL) {
        printf("memory allocation failed\n");
        return 0;
    }

    net_index = htonl(msg->index);

    memcpy(sendbuf + pos, &net_index, 4);
    pos += 4;

    memcpy(sendbuf + pos, msg->phone1, 12);
    pos += 12;

    memcpy(sendbuf + pos, msg->phone2, 12);
    pos += 12;

    sendbuf[pos++] = (char)msg->hour;
    sendbuf[pos++] = (char)msg->minute;
    sendbuf[pos++] = (char)msg->second;

    memcpy(sendbuf + pos, text, text_len);
    pos += text_len;

    sendbuf[pos] = '\0';

    result = SendAll(s, sendbuf, (int)msg_size);
    free(sendbuf);
    return result;
}

// Читаем одну строку файла целиком, даже если она длиннее обычного
буфера.
int ReadLongLine(FILE *f, char **line_out) {
    char part[4096];
    char *line = NULL;
    size_t used = 0;

    *line_out = NULL;

    while (fgets(part, sizeof(part), f) != NULL) {
        size_t part_len = strlen(part);
        char *new_line = (char *)realloc(line, used + part_len +
1);

        if (new_line == NULL) {
            free(line);
            return 0;
        }

        line = new_line;
        memcpy(line + used, part, part_len + 1);
        used += part_len;
```

```
        if (used > 0 && line[used - 1] == '\n') {
            break;
        }
    }

    if (line == NULL) {
        return 0;
    }

    *line_out = line;
    return 1;
}

// Общее завершение клиента: закрываем все ресурсы и возвращаем код выхода.
int Finish(FILE *f, SOCKET s, int code) {
    if (s != INVALID_SOCKET) {
        closesocket(s);
    }
    if (f != NULL) {
        fclose(f);
    }
    WSACleanup();
    return code;
}

int main(int argc, char *argv[]) {
    FILE *f = NULL;                 // Входной файл с текстовыми сообщениями.
    SOCKET s = INVALID_SOCKET;   // Сокет клиента для связи с сервером.
    unsigned int index = 0;      // Номер текущего сообщения по протоколу.

    if (argc != 3) {
        printf("Usage: tcpclient <ip:port> <file>\n");
        return 1;
    }

    f = fopen(argv[2], "rb");
    if (f == NULL) {
        printf("Cannot open file\n");
        return 1;
    }

    {
        WSADATA wsa; // Служебная структура WinSock.
        if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
            printf("WSAStartup failed\n");
            return Finish(f, s, 1);
        }
    }
```

```
    {
        char server_text[256];   // Строка вида 127.0.0.1:9000
        char *colon = NULL;      // Указатель на символ ':' внутри
строки адреса.
        char *end = NULL;           // Нужен для strtol, чтобы
проверить корректность порта.
        long port = 0;           // Числовое значение порта после
разбора строки.
        sockaddr_in server_addr; // Структура адреса сервера для
connect.
        int attempt;             // Номер попытки подключения.

        // Копируем адрес сервера в свой буфер, чтобы можно было
безопасно менять строку.
        strncpy(server_text, argv[1], sizeof(server_text) - 1);
        server_text[sizeof(server_text) - 1] = '\0';

        // Ищем двоеточие между IP и портом.
        colon = strchr(server_text, ':');
        if (colon == NULL) {
            printf("Invalid server address\n");
            return Finish(f, s, 1);
        }

        // Разрезаем строку на две части:
        // server_text -> IP
        // colon       -> строка порта
        *colon = '\0';
        ++colon;

        if (server_text[0] == '\0' || colon[0] == '\0') {
            printf("Invalid server address\n");
            return Finish(f, s, 1);
        }

        // Преобразуем порт из текста в число.
        port = strtol(colon, &end, 10);
        if (*end != '\0' || port <= 0 || port > 65535) {
            printf("Invalid server address\n");
            return Finish(f, s, 1);
        }

        // Заполняем адрес сервера для будущего connect.
        memset(&server_addr, 0, sizeof(server_addr));
        server_addr.sin_family = AF_INET;
        server_addr.sin_port = htons((unsigned short)port);
        server_addr.sin_addr.s_addr = inet_addr(server_text);
        if (server_addr.sin_addr.s_addr == INADDR_NONE) {
            printf("Invalid server address\n");
            return Finish(f, s, 1);
        }
```

```
        // По условию делаем до 10 попыток подключения с паузой
100 мс.
        // Каждый раз создаём новый сокет и пробуем connect.
        for (attempt = 0; attempt < 10; ++attempt) {
            s = socket(AF_INET, SOCK_STREAM, 0);
            if (s == INVALID_SOCKET) {
                Sleep(100);
                continue;
            }

            if     (connect(s,     (sockaddr     *)&server_addr,
sizeof(server_addr)) == 0) {
                break;
            }

            closesocket(s);
            s = INVALID_SOCKET;
            Sleep(100);
        }

        if (s == INVALID_SOCKET) {
            printf("connect failed\n");
            return Finish(f, s, 1);
        }
    }

    // По протоколу сначала отправляем команду "put":
    // это говорит серверу, что дальше пойдут сообщения.
    if (!SendAll(s, "put", 3)) {
        return Finish(f, s, 1);
    }

    while (1) {
        Message msg;            // Разобранное сообщение в удобном
виде.
        char *line = NULL;       // Одна строка из входного файла
любой длины.
        const char *text;        // Указатель на начало текстовой
части сообщения.
        int hh;                  // Часы как обычное число int.
        int mm;                  // Минуты как обычное число int.
        int ss;                  // Секунды как обычное число int.
        char ok[2];              // Сюда читаем ответ сервера: 'o'
'k'.
        size_t len;              // Длина строки line.

        // Читаем очередную строку из файла. Если файл закончился,
выходим из цикла.
        if (!ReadLongLine(f, &line)) break;
        if (line[0] == '\0' || line[0] == '\n' || line[0] == '\r')
{
            free(line);
            continue;
```

```
        }

        // Убираем перевод строки в конце, если он есть.
        len = strlen(line);
        while (len > 0 && (line[len - 1] == '\n' || line[len - 1]
== '\r')) {
            line[len - 1] = '\0';
            --len;
        }

        // Минимально проверяем формат:
        // +7xxxxxxxxxx +7xxxxxxxxxx hh:mm:ss text
        if (line[0] == '\0') {
            free(line);
            continue;
        }
        if (strlen(line) < 36) {
            free(line);
            continue;
        }
        if (line[12] != ' ' || line[25] != ' ' || line[34] != ' ')
{
            free(line);
            continue;
        }

        // Копируем телефоны из строки в структуру сообщения.
        memcpy(msg.phone1, line, 12);
        memcpy(msg.phone2, line + 13, 12);
        msg.phone1[12] = '\0';
        msg.phone2[12] = '\0';

        if (!CheckPhone(msg.phone1) || !CheckPhone(msg.phone2)) {
            free(line);
            continue;
        }

        // Проверяем, что время действительно записано цифрами и
двоеточиями.
        if (line[28] != ':' || line[31] != ':') {
            free(line);
            continue;
        }
        if (line[26] < '0' || line[26] > '9') {
            free(line);
            continue;
        }
        if (line[27] < '0' || line[27] > '9') {
            free(line);
            continue;
        }
        if (line[29] < '0' || line[29] > '9') {
            free(line);
```

```
            continue;
        }
        if (line[30] < '0' || line[30] > '9') {
            free(line);
            continue;
        }
        if (line[32] < '0' || line[32] > '9') {
            free(line);
            continue;
        }
        if (line[33] < '0' || line[33] > '9') {
            free(line);
            continue;
        }

        // Переводим время из текстового вида "hh:mm:ss" в обычные
числа.
        hh = (line[26] - '0') * 10 + (line[27] - '0');
        mm = (line[29] - '0') * 10 + (line[30] - '0');
        ss = (line[32] - '0') * 10 + (line[33] - '0');

        if (!CheckTime(hh, mm, ss)) {
            free(line);
            continue;
        }

        // Копируем разобранные поля в структуру сообщения.
        msg.hour = (unsigned char)hh;
        msg.minute = (unsigned char)mm;
        msg.second = (unsigned char)ss;
        text = line + 35;

        if (!CheckText(text)) {
            free(line);
            continue;
        }

        msg.index = index;

        // Как и в Ruby-эмуляторе клиента, собираем всё сообщение
в один буфер
        // и только потом отправляем его на сервер.
        if (!SendMessage(s, &msg, text)) {
            free(line);
            return Finish(f, s, 1);
        }

        // После каждого сообщения ждём от сервера ответ "ok".
        if (!RecvAll(s, ok, 2)) {
            free(line);
            return Finish(f, s, 1);
        }
```

```
        if (ok[0] != 'o' || ok[1] != 'k') {
            printf("Invalid confirmation\n");
            free(line);
            return Finish(f, s, 1);
        }

        free(line);
        ++index;
    }

    return Finish(f, s, 0);
}
```

Приложение Б «tcpclient2.cpp»

```
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <fstream>
#include <string>

#pragma comment(lib, "ws2_32.lib")

struct Message {
    char phone1[13];          // Первый номер телефона. 13, а не
12, чтобы влез '\0'.
    char phone2[13];         // Второй номер телефона.
    unsigned char hour;      // Часы.
    unsigned char minute;    // Минуты.
    unsigned char second;    // Секунды.
    std::string text;        // Текст сообщения любой длины.
};

static int init_winsock() {
    WSADATA data;
    int err = WSAStartup(MAKEWORD(2, 2), &data);
    if (err != 0) {
        printf("WSAStartup failed: %d\n", err);
        return 0;
    }
    return 1;
}

static void cleanup_socket(SOCKET s) {
    // Общее завершение: закрываем сокет и выгружаем WinSock.
    if (s != INVALID_SOCKET) {
        closesocket(s);
    }
    WSACleanup();
}

static int parse_server_address(const char *arg, sockaddr_in
*addr, std::string *server_prefix) {
    // Разбираем строку вида "IP:port" и готовим sockaddr_in для
connect.
    // Дополнительно собираем строку "IP:port ", которая потом
пойдёт в начало
    // каждой строки выходного файла.
    std::string addr_port;
    size_t colon_pos;
    std::string ip_part;
    std::string port_part;
```

```
    long port_value;
    char *end = NULL;
    unsigned long ip_value;

    if (arg == NULL || addr == NULL) {
        return 0;
    }

    addr_port = arg;
    colon_pos = addr_port.find(':');
    if (colon_pos == std::string::npos) {
        return 0;
    }

    ip_part = addr_port.substr(0, colon_pos);
    port_part = addr_port.substr(colon_pos + 1);
    if (ip_part.empty() || port_part.empty()) {
        return 0;
    }

    port_value = strtol(port_part.c_str(), &end, 10);
    if (*end != '\0' || port_value <= 0 || port_value > 65535) {
        return 0;
    }

    ip_value = inet_addr(ip_part.c_str());
    if (ip_value == INADDR_NONE) {
        return 0;
    }

    memset(addr, 0, sizeof(*addr));
    addr->sin_family = AF_INET;
    addr->sin_port = htons((unsigned short)port_value);
    addr->sin_addr.s_addr = ip_value;

    if (server_prefix != NULL) {
        *server_prefix = ip_part + ":" + port_part + " ";
    }

    return 1;
}

static int connect_with_retry(SOCKET s, const sockaddr_in *addr)
{
    int attempt;

    // По условию делаем до 10 попыток подключения с паузой 100
мс.
    for (attempt = 0; attempt < 10; ++attempt) {
        if (connect(s, (const sockaddr *)addr, sizeof(*addr)) !=
SOCKET_ERROR) {
            return 1;
        }
```

```
        Sleep(100);
    }

    printf("Failed to connect\n");
    return 0;
}

static int set_socket_timeouts(SOCKET s) {
    // Таймауты защищают от бесконечного зависания на send/recv.
    int timeout_ms = 5000;

    if (setsockopt(s, SOL_SOCKET, SO_RCVTIMEO, (const char
*)&timeout_ms, sizeof(timeout_ms)) == SOCKET_ERROR) {
        return 0;
    }
    if (setsockopt(s, SOL_SOCKET, SO_SNDTIMEO, (const char
*)&timeout_ms, sizeof(timeout_ms)) == SOCKET_ERROR) {
        return 0;
    }

    return 1;
}

static int send_all(SOCKET s, const char *data, int size) {
    int total = 0;

    // send может отправить только часть буфера, поэтому досылаем
в цикле.
    while (total < size) {
        int sent = send(s, data + total, size - total, 0);
        if (sent == SOCKET_ERROR || sent == 0) {
            printf("send failed: %d\n", WSAGetLastError());
            return 0;
        }
        total += sent;
    }

    return 1;
}

static int recv_all(SOCKET s, char *buffer, int size) {
    int total = 0;

    // recv тоже может вернуть не все байты сразу, поэтому
дочитываем до конца.
    while (total < size) {
        int received = recv(s, buffer + total, size - total, 0);
        if (received == SOCKET_ERROR || received == 0) {
            return 0;
        }
        total += received;
    }
```

```
    return 1;
}

static int recv_message(SOCKET s, Message *msg) {
    // Сервер при get отправляет сообщение в бинарном формате:
    // 4 байта index, 12 байт phone1, 12 байт phone2, 3 байта
времени,
    // затем текст до '\0'.
    unsigned int net_index;
    char time_bytes[3];
    char ch;

    if (!recv_all(s, (char *)&net_index, 4)) {
        return 0;
    }

    if (!recv_all(s, msg->phone1, 12)) {
        return 0;
    }
    msg->phone1[12] = '\0';

    if (!recv_all(s, msg->phone2, 12)) {
        return 0;
    }
    msg->phone2[12] = '\0';

    if (!recv_all(s, time_bytes, 3)) {
        return 0;
    }

    msg->hour = (unsigned char)time_bytes[0];
    msg->minute = (unsigned char)time_bytes[1];
    msg->second = (unsigned char)time_bytes[2];
    msg->text.clear();

    // Текст заранее неизвестной длины, поэтому читаем его
посимвольно
    // до завершающего нулевого байта.
    while (1) {
        int received = recv(s, &ch, 1, 0);
        if (received == SOCKET_ERROR || received == 0) {
            return 0;
        }
        if (ch == '\0') {
            break;
        }
        msg->text.push_back(ch);
    }

    return 1;
}
```

```
static void write_message(FILE *file, const std::string
&server_prefix, const Message *msg) {
    // В выходной файл номер сообщения не записывается.
    // Сохраняем только адрес сервера, телефоны, время и текст.
    fprintf(file,
            "%s%s %s %02u:%02u:%02u %s\n",
            server_prefix.c_str(),
            msg->phone1,
            msg->phone2,
            (unsigned int)msg->hour,
            (unsigned int)msg->minute,
            (unsigned int)msg->second,
            msg->text.c_str());
}

int main(int argc, char *argv[]) {
    sockaddr_in server_addr;     // Сетевой адрес сервера.
    std::string server_prefix;    // Строка вида "IP:port " для
записи в файл.
    SOCKET s = INVALID_SOCKET;   // TCP-сокет клиента.
    FILE *file = NULL;           // Выходной файл, куда сохраняем
полученные сообщения.
    Message msg;                 // Очередное сообщение, принятое
от сервера.

    if (argc < 4) {
        printf("Invalid args\n");
        return 1;
    }

    // Второй аргумент должен быть ровно "get".
    if (strcmp(argv[2], "get") != 0) {
        printf("Invalid command\n");
        return 1;
    }

    if       (!parse_server_address(argv[1],       &server_addr,
&server_prefix)) {
        printf("Invalid server address\n");
        return 1;
    }

    // Сначала просто проверяем, что выходной файл можно
создать/очистить.
    file = fopen(argv[3], "wb");
    if (file == NULL) {
        printf("Cannot open file\n");
        return 1;
    }
    fclose(file);

    if (!init_winsock()) {
        return 1;
```

```
    }

    s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (s == INVALID_SOCKET) {
        printf("socket failed: %d\n", WSAGetLastError());
        cleanup_socket(s);
        return 1;
    }

    if (!set_socket_timeouts(s)) {
        printf("cannot set socket timeouts\n");
        cleanup_socket(s);
        return 1;
    }

    if (!connect_with_retry(s, &server_addr)) {
        cleanup_socket(s);
        return 1;
    }

    // Команда "get" говорит серверу, что нужно вернуть все
сообщения из msg.txt.
    if (!send_all(s, "get", 3)) {
        cleanup_socket(s);
        return 1;
    }

    // Открываем файл повторно уже в режиме добавления и начинаем
сохранять данные.
    file = fopen(argv[3], "ab");
    if (file == NULL) {
        printf("Cannot open file\n");
        cleanup_socket(s);
        return 1;
    }

    // Сервер сам закроет соединение после передачи последнего
сообщения.
    // Пока сообщения читаются успешно, сохраняем их в выходной
файл.
    while (recv_message(s, &msg)) {
        write_message(file, server_prefix, &msg);
    }

    fclose(file);
    cleanup_socket(s);
    return 0;
}
```

Приложение В «tcpserver.cpp»

```
#include <arpa/inet.h>
#include <errno.h>
#include <fcntl.h>
#include <netinet/in.h>
#include <sys/select.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cstdint>
#include <string>
#include <vector>

#define MAX_CLIENTS 1024      // сколько максимум клиентов можем
держать
#define RECV_CHUNK_SIZE 4096 // сколько байт читаем за один recv
#define FIXED_PART_SIZE 31   // размер обязательной части сообщения

enum ClientMode {
    MODE_WAIT_COMMAND = 0,
    MODE_PUT = 1,
    MODE_GET = 2
};

struct Client {
    int fd;                     // сокет клиента
    sockaddr_in addr;           // адрес клиента
    int mode;                   // put/get или уже работаем
    int close_after_send;       // надо ли закрыть после отправки
(для get)
    int stop_after_send;        // надо ли завершить сервер после
отправки
    int peer_closed;              // клиент закрыл соединение(для
put)
    std::vector<char> in_buf;  // Всё, что пришло от клиента
    std::vector<char> out_buf; // Всё, что сервер хочет отправить
клиенту( для get)
    size_t out_sent;              // сколько из out_buf удалось
отправить
};

static int CheckPort(const char *text, int *port_out) {   //
Проверяем порт и если он верный то возвращаем его
    char *end = NULL;
    long port = 0;

    if (text == NULL || *text == '\0') return 0;

    port = strtol(text, &end, 10);
```

```
    if (*end != '\0') return 0;
    if (port <= 0 || port > 65535) return 0;

    *port_out = (int)port;
    return 1;
}

static int SetNonBlocking(int fd) {
    // fcntl — это системная функция Linux для работы с настройками
дескриптора.
    int flags = fcntl(fd, F_GETFL, 0); // F_GETFL - получить
текущее состояние флагов этого дескриптора
    if (flags < 0) return 0;
    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0) return 0; //
Устанавливаем неблокирующий режим - если операция сейчас
невозможна, программа не ждёт, а сразу возвращается
    return 1;
}

static void InitClient(Client *c) {            // Инициализация
клиента
    c->fd = -1;
    memset(&c->addr, 0, sizeof(c->addr));
    c->mode = MODE_WAIT_COMMAND;
    c->close_after_send = 0;
    c->stop_after_send = 0;
    c->peer_closed = 0;
    c->in_buf.clear();
    c->out_buf.clear();
    c->out_sent = 0;
}

static void CloseClient(Client *c) {
    if (c->fd >= 0) {
        close(c->fd);
    }
    InitClient(c);
}

static int FlushOutput(Client *c) { // Сервер отправляет клиенту
то , что лежит в out_buf например "ok"
    while (c->out_sent < c->out_buf.size()) {
        ssize_t sent = send(c->fd,
                            &c->out_buf[c->out_sent],
                            c->out_buf.size() - c->out_sent,
                            0);
        if (sent > 0) {
            c->out_sent += (size_t)sent;
            continue;
        }
        if (sent < 0 && (errno == EAGAIN || errno == EWOULDBLOCK))
{
            return 1;
```

```
        }
        return 0;
    }

    if (c->out_sent >= c->out_buf.size()) {
        c->out_buf.clear();
        c->out_sent = 0;
    }
    return 1;
}

static int QueueOk(Client *c) {
    c->out_buf.push_back('o');
    c->out_buf.push_back('k');
    return 1;
}

static int ValidText(const char *text, size_t len) {
    size_t i;
    if (len == 0) return 0;
    for (i = 0; i < len; ++i) {
        if (text[i] == '\n' || text[i] == '\r') return 0;
    }
    return 1;
}

static int WriteMessageToFile(const Client *c,
                              const char *phone1,
                              const char *phone2,
                              unsigned char hh,
                              unsigned char mm,
                              unsigned char ss,
                              const char *text,
                              size_t text_len) {
    FILE *f = fopen("msg.txt", "ab");
    if (f == NULL) {
        printf("Error: cannot open msg.txt\n");
        return 0;
    }

    fprintf(f,
            "%s:%u %.*s %.*s %02u:%02u:%02u ",
            inet_ntoa(c->addr.sin_addr),
            (unsigned int)ntohs(c->addr.sin_port),
            12,
            phone1,
            12,
            phone2,
            (unsigned int)hh,
            (unsigned int)mm,
            (unsigned int)ss);

    if (text_len > 0) {
```

```
        fwrite(text, 1, text_len, f);
    }
    fputc('\n', f);
    fclose(f);
    return 1;
}

static int QueueGetMessage(Client *c,
                           uint32_t idx,
                           const char *phone1,
                           const char *phone2,
                           unsigned char hh,
                           unsigned char mm,
                           unsigned char ss,
                           const char *text) {
    uint32_t net_idx = htonl(idx);
    char p1[12];
    char p2[12];
    size_t len1 = strlen(phone1);
    size_t len2 = strlen(phone2);

    memset(p1, ' ', sizeof(p1));
    memset(p2, ' ', sizeof(p2));

    if (len1 > 12) len1 = 12;
    if (len2 > 12) len2 = 12;
    memcpy(p1, phone1, len1);
    memcpy(p2, phone2, len2);

    c->out_buf.insert(c->out_buf.end(), (char *)&net_idx, (char
*)&net_idx + 4);
    c->out_buf.insert(c->out_buf.end(), p1, p1 + 12);
    c->out_buf.insert(c->out_buf.end(), p2, p2 + 12);
    c->out_buf.push_back((char)hh);
    c->out_buf.push_back((char)mm);
    c->out_buf.push_back((char)ss);
    c->out_buf.insert(c->out_buf.end(),     text,     text     +
strlen(text));
    c->out_buf.push_back('\0');
    return 1;
}

static int LoadHistoryForGet(Client *c) {
    FILE *f = fopen("msg.txt", "rb");
    char line[65536];
    uint32_t idx = 0;

    if (f == NULL) return 1;

    while (fgets(line, sizeof(line), f) != NULL) {
        char *nl = strchr(line, '\n');
        char peer_ignored[128];
        char phone1[64];
```

```
        char phone2[64];
        char tstr[64];
        char text[65000];
        int hh, mm, ss;

        if (nl != NULL) *nl = '\0';
        text[0] = '\0';

        if (sscanf(line, "%127s %63s %63s %63s %64999[^\n]",
                   peer_ignored, phone1, phone2, tstr, text) < 4)
{
            continue;
        }

        if  (sscanf(tstr,  "%d:%d:%d",  &hh,  &mm,  &ss)  !=  3)
continue;
        if (hh < 0 || hh > 23 || mm < 0 || mm > 59 || ss < 0 || ss
> 59) continue;

        QueueGetMessage(c,
                        idx,
                        phone1,
                        phone2,
                        (unsigned char)hh,
                        (unsigned char)mm,
                        (unsigned char)ss,
                        text);
        ++idx;
    }

    fclose(f);
    return 1;
}

int main(int argc, char *argv[]) {  // tcpserver <port>
    int port = 0;
    int listen_fd = -1;            // дескриптор серверного сокета
    Client  clients[MAX_CLIENTS];      // сколько клиентов может
подключиться к серверу
    int  shutdown_requested  =  0;       // Нужно ли останавливать
сервер? 0 - значит нет пока
    int i;

    if (argc < 2) {
        printf("Error: port is not specified\n");
        return 1;
    }
    if (!CheckPort(argv[1], &port)) {
        printf("Error: invalid port\n");
        return 1;
    }
```

```
    listen_fd = socket(AF_INET, SOCK_STREAM, 0); // Создаём tcp-
сокет
    if (listen_fd < 0) {
        printf("Error: cannot create socket\n");
        return 1;
    }


    int reuse = 1;
    sockaddr_in saddr;

    // SO_REUSEADDR опция разрешает сокету повторно использовать
адрес/порт быстрее
    // SOL_SOCKET - общие опции сокета, для настройки SO_REUSEADDR
    setsockopt(listen_fd,   SOL_SOCKET,   SO_REUSEADDR,   &reuse,
sizeof(reuse));

    if (!SetNonBlocking(listen_fd)) {
        printf("Error: cannot set nonblocking mode\n");
        close(listen_fd);
        return 1;
    }

    memset(&saddr, 0, sizeof(saddr));
    saddr.sin_family = AF_INET;                    // используем
семейство адресов IPv4
    saddr.sin_addr.s_addr = htonl(INADDR_ANY);   // любой адрес
слушаем (в сетевом порядке байт)
    saddr.sin_port = htons((uint16_t)port);       // порт, к-ый
сервер будет слушать

    if (bind(listen_fd, (sockaddr *)&saddr, sizeof(saddr)) < 0) {
// привязываем сокет к адресу и порту
        printf("Error:  cannot  bind  port  %d:  %s\n",  port,
strerror(errno));
        close(listen_fd);
        return 1;
    }

    if (listen(listen_fd, SOMAXCONN) < 0) {    // делаем серверный
сокет слушащим, SOMAXCONN - макс длина очереди ожидающих
подключений
        printf("Error: cannot listen\n");
        close(listen_fd);
        return 1;
    }


    for (i = 0; i < MAX_CLIENTS; ++i) {    // инициализируем весь
массив клиентов
        InitClient(&clients[i]);
    }
```

```
    while (!shutdown_requested) {
        fd_set read_set;  // сокеты из к-ых можно читать
        fd_set write_set; // сокеты в к-ые можно писать(для get)
        int max_fd = listen_fd; // максимальный номер дескриптора
        int ready = 0;

        FD_ZERO(&read_set);  //  очищаем  множество  read_set  и
write_set
        FD_ZERO(&write_set);
        FD_SET(listen_fd, &read_set); // listen_fd в read_set

        for (i = 0; i < MAX_CLIENTS; ++i) {
            /*  в  каждом  элементе  массива  сидит  либо  реально
подключённый клиент,
               либо слот пустой */
            if (clients[i].fd < 0) continue;
            FD_SET(clients[i].fd, &read_set);
            if  (clients[i].out_sent  <  clients[i].out_buf.size())
{
                FD_SET(clients[i].fd, &write_set);
            }

            if (clients[i].fd > max_fd) { // это нужно для select
                max_fd = clients[i].fd;
            }
        }

        ready = select(max_fd + 1, &read_set, &write_set, NULL,
NULL);
        if (ready < 0) {
            if (errno == EINTR) continue;
            printf("Error: select failed\n");
            break;
        }

        if (FD_ISSET(listen_fd, &read_set)) { // сервер принимает
новые подключения клиентов
            while (1) { // новых подключений может быть несколько
                sockaddr_in caddr; // подготовка адреса клиента
                socklen_t caddr_len = sizeof(caddr);
                int  cfd  =  accept(listen_fd,  (sockaddr  *)&caddr,
&caddr_len); // сервер принимает 1 подключение

                if (cfd < 0) {
                    if (errno == EAGAIN || errno == EWOULDBLOCK)
break; // новых подключений больше нет
                    printf("Error: accept failed\n");
                    break;
                }

                if (!SetNonBlocking(cfd)) {
                    close(cfd);
                    continue;
```

```
                }

                for (i = 0; i < MAX_CLIENTS; ++i) {
                    if (clients[i].fd < 0) {
                        clients[i].fd = cfd;
                        clients[i].addr = caddr;
                        clients[i].mode = MODE_WAIT_COMMAND;
                        clients[i].close_after_send = 0;
                        clients[i].stop_after_send = 0;
                        clients[i].peer_closed = 0;
                        break;
                    }
                }

                if (i == MAX_CLIENTS) { // свободного места для
клиента не найдено
                    close(cfd);
                }
            }
        }

        for (i = 0; i < MAX_CLIENTS; ++i) {
            if (clients[i].fd < 0) continue;

            if (FD_ISSET(clients[i].fd, &read_set)) {
                while (1) { // пока из клиентского сокета можно
читать данные
                    char temp[RECV_CHUNK_SIZE]; // порция данных
пришедших от клиента
                    ssize_t   r   =   recv(clients[i].fd,   temp,
sizeof(temp), 0); // читаем данные от клиента

                    if (r > 0) {

clients[i].in_buf.insert(clients[i].in_buf.end(),  temp,  temp  +
r);
                        continue;
                    }
                    if (r == 0) { // клиент закрыл соединение
                        clients[i].peer_closed = 1;
                        break;
                    }
                    if (errno == EAGAIN || errno == EWOULDBLOCK)
{ // на данный момент данных больше нет
                        break;
                    }
                    CloseClient(&clients[i]);   //   если   другая
ошибка, закрываем клиента.
                    break;
                }

                if (clients[i].fd < 0) continue;
```

```
                if (clients[i].mode == MODE_WAIT_COMMAND &&
clients[i].in_buf.size() >= 3) { // Разбираем первую команду
клиента
                    if (memcmp(&clients[i].in_buf[0], "put", 3)
== 0) {
                        clients[i].mode = MODE_PUT;
                    } else if (memcmp(&clients[i].in_buf[0],
"get", 3) == 0) {
                        clients[i].mode = MODE_GET;
                    } else {
                        printf("Invalid command\n");
                        CloseClient(&clients[i]);
                        continue;
                    }

clients[i].in_buf.erase(clients[i].in_buf.begin(),
clients[i].in_buf.begin() + 3);

                    if (clients[i].mode == MODE_GET) { // Если
режим GET
                        if (!LoadHistoryForGet(&clients[i])) {
                            CloseClient(&clients[i]);
                            continue;
                        }
                        clients[i].close_after_send = 1;
                    }
                }

                if (clients[i].fd < 0) continue;

                if (clients[i].mode == MODE_PUT) {
                    while (1) { // пока во входном буфере есть
полные сообщения, разбирать их одно за другим
                        size_t zero_pos = FIXED_PART_SIZE; //
конец текста сообщения, позиция
                        char phone1[13];
                        char phone2[13];
                        unsigned char hh;
                        unsigned char mm;
                        unsigned char ss;
                        const char *text_ptr;
                        size_t text_len;

                        if (clients[i].in_buf.size() <
FIXED_PART_SIZE) break; // некорректное сообщение

                        while (zero_pos <
clients[i].in_buf.size() && // ищем конец текста
                               clients[i].in_buf[zero_pos] !=
'\0') {
                            ++zero_pos;
                        }
```

```
                        if (zero_pos == clients[i].in_buf.size())
break; // не нашли терминальный нуль

                        memcpy(phone1,     &clients[i].in_buf[4],
12);
                        memcpy(phone2,    &clients[i].in_buf[16],
12);
                        phone1[12] = '\0';
                        phone2[12] = '\0';
                        hh                =                (unsigned
char)clients[i].in_buf[28];
                        mm                =                (unsigned
char)clients[i].in_buf[29];
                        ss                =                (unsigned
char)clients[i].in_buf[30];
                        text_ptr                                    =
&clients[i].in_buf[FIXED_PART_SIZE];
                        text_len = zero_pos - FIXED_PART_SIZE;

                        if (hh <= 23 && mm <= 59 && ss <= 59 &&
ValidText(text_ptr, text_len)) {
                            if  (!WriteMessageToFile(&clients[i],
phone1, phone2, hh, mm, ss, text_ptr, text_len)) {
                                CloseClient(&clients[i]);
                                break;
                            }
                            QueueOk(&clients[i]);
                            if (text_len == 4 && memcmp(text_ptr,
"stop", 4) == 0) {
                                clients[i].stop_after_send = 1;
                            }
                        }


clients[i].in_buf.erase(clients[i].in_buf.begin(),

clients[i].in_buf.begin() + zero_pos + 1);
                    }
                }
            }

            if  (clients[i].fd  >=  0  &&  FD_ISSET(clients[i].fd,
&write_set)) {
                if (!FlushOutput(&clients[i])) {
                    CloseClient(&clients[i]);
                    continue;
                }
            }

            if (clients[i].fd >= 0 &&
                clients[i].stop_after_send &&
                clients[i].out_buf.empty()) {
                shutdown_requested = 1;
```

```
            }

            if (clients[i].fd >= 0 &&
                clients[i].close_after_send &&
                clients[i].out_buf.empty()) {
                CloseClient(&clients[i]);
                continue;
            }

            if (clients[i].fd >= 0 &&
                clients[i].peer_closed &&
                clients[i].out_buf.empty()) {
                CloseClient(&clients[i]);
            }
        }
    }

    for (i = 0; i < MAX_CLIENTS; ++i) {
        if (clients[i].fd >= 0) {
            CloseClient(&clients[i]);
        }
    }

    close(listen_fd);
    return 0;
}
```

Приложение Г «udpclient.cpp»

```
#include <arpa/inet.h>   // inet_pton, htons, htonl, ntohl
#include <netinet/in.h>  // sockaddr_in
#include <sys/socket.h>  // socket, sendto, recvfrom
#include <sys/select.h>  // select
#include <unistd.h>      // close

#include <iostream>
#include <cstring>       // strchr, strlen, memcpy
#include <cstdlib>       // strtol, malloc, realloc, free
#include <cstdio>
#include <cstdint>
#include <sys/types.h>

struct Message {
    uint32_t index;    // Номер сообщения во входном файле.
    char phone1[13];    // Первый телефон. 13, а не 12, чтобы
добавить '\0'.
    char phone2[13];   // Второй телефон.
    uint8_t hour;      // Часы.
    uint8_t minute;    // Минуты.
    uint8_t second;    // Секунды.
};

struct PreparedMessage {
    uint32_t index;                // Номер сообщения, которому
соответствует дейтаграмма.
    unsigned char* datagram;  // Уже собранный бинарный пакет.
    size_t datagram_len;      // Длина пакета в байтах.
};

bool BuildDatagram(const Message* msg, const char* text, unsigned
char** out_buf, size_t* out_len) {
    // Собираем одну UDP-дейтаграмму в формате протокола:
    // index(4) + phone1(12) + phone2(12) + hh(1) + mm(1) + ss(1)
+ text + '\0'
    if (text == nullptr) return false;

    size_t text_len = strlen(text);
    size_t packet_len = 4 + 12 + 12 + 3 + text_len + 1;
    unsigned char* packet = (unsigned char*)malloc(packet_len);
    if (packet == nullptr) return false;

    size_t offset = 0;
    uint32_t net_index = htonl(msg->index);

    memcpy(packet + offset, &net_index, 4);
    offset += 4;

    memcpy(packet + offset, msg->phone1, 12);
    offset += 12;

    memcpy(packet + offset, msg->phone2, 12);
```

```
    offset += 12;

    packet[offset++] = msg->hour;
    packet[offset++] = msg->minute;
    packet[offset++] = msg->second;

    memcpy(packet + offset, text, text_len);
    offset += text_len;

    packet[offset++] = '\0';

    *out_buf = packet;
    *out_len = offset;
    return true;
}

bool ParseServerAddress(const char* s, sockaddr_in* out_addr) {
    // Разбираем строку вида "IP:port" и заполняем sockaddr_in для
sendto.
    if (s == nullptr || out_addr == nullptr) {
        return false;
    }

    const char* pos = strchr(s, ':');
    if (pos == nullptr) {
        return false;
    }

    char ip[INET_ADDRSTRLEN];
    size_t ip_len = pos - s;
    if (ip_len == 0 || ip_len >= INET_ADDRSTRLEN) {
        return false;
    }
    memcpy(ip, s, ip_len);
    ip[ip_len] = '\0';

    char* end = nullptr;
    long port = strtol(pos + 1, &end, 10);
    if (end == pos + 1 || *end != '\0' || port < 1 || port > 65535)
{
        return false;
    }

    if (inet_pton(AF_INET, ip, &out_addr->sin_addr) != 1) {
        return false;
    }

    out_addr->sin_family = AF_INET;
    out_addr->sin_port = htons((uint16_t)port);
    return true;
}

int main(int argc, char* argv[]) {
```

```
    if (argc != 3) {
        std::cerr << "Usage: udpclient <ip:port> <file>\n";
        return 1;
    }

    sockaddr_in serv_addr{}; // Адрес сервера, куда будут
отправляться все дейтаграммы.

    const char* arg_addr = argv[1];
    if (!ParseServerAddress(arg_addr, &serv_addr)) {
        std::cerr << "Invalid server address\n";
        return 1;
    }

    int s = socket(AF_INET, SOCK_DGRAM, 0);
    if (s < 0) {
        std::cerr << "Error creation client socket\n";
        return 1;
    }

    FILE* f = fopen(argv[2], "r");
    if (f == nullptr) {
        std::cerr << "Cannot open file\n";
        close(s);
        return 1;
    }

    char* line = nullptr;   // Сюда getline будет класть очередную
строку файла.
    size_t capacity = 0;    // Текущий размер буфера line.
    ssize_t read_len = 0;   // Длина считанной строки.

    uint32_t current_index = 0; // Номер текущего сообщения по
протоколу.

    PreparedMessage* msgs = NULL; // Массив всех подготовленных
дейтаграмм.
    size_t msg_count = 0;          // Сколько сообщений удалось
подготовить.

    while ((read_len = getline(&line, &capacity, f)) != -1) {
        // Убираем перевод строки в конце, чтобы потом не
отправлять его серверу.
        while (read_len > 0 && (line[read_len - 1] == '\n' ||
line[read_len - 1] == '\r')) {
            line[--read_len] = '\0';
        }

        // Пустые и слишком короткие строки считаем некорректными
и пропускаем.
        if (read_len == 0) continue;
        if (read_len < 35) continue;
```

```
        Message msg{};
        msg.index = current_index;

        // Копируем телефоны из входной строки.
        memcpy(msg.phone1, line, 12);
        msg.phone1[12] = '\0';

        memcpy(msg.phone2, line + 13, 12);
        msg.phone2[12] = '\0';

        // Переводим время из текстового вида "hh:mm:ss" в три
отдельных числа.
        msg.hour = (line[26] - '0') * 10 + (line[27] - '0');
        msg.minute = (line[29] - '0') * 10 + (line[30] - '0');
        msg.second = (line[32] - '0') * 10 + (line[33] - '0');

        const char* text_ptr = line + 35;    // Текст начинается
после "phone1 phone2 hh:mm:ss ".
        unsigned char* datagram = nullptr;   // Готовый бинарный
пакет.
        size_t datagram_len = 0;             // Его длина.
        if     (!BuildDatagram(&msg,     text_ptr,     &datagram,
&datagram_len)) {
            continue;
        }

        //  Добавляем собранную дейтаграмму в общий массив
сообщений.
        PreparedMessage*                new_msgs                =
(PreparedMessage*)realloc(msgs,     (msg_count     +     1)     *
sizeof(PreparedMessage));
        if (new_msgs == NULL) {
            free(datagram);
            break;
        }
        msgs = new_msgs;

        msgs[msg_count].index = msg.index;
        msgs[msg_count].datagram = datagram;
        msgs[msg_count].datagram_len = datagram_len;
        msg_count++;

        current_index++;
    }

    // Сервер в ответ возвращает не больше 20 последних id, поэтому
клиент ждёт
    // подтверждения либо по всем сообщениям, либо максимум по 20.
    size_t target;
    if (msg_count < 20) {
        target = msg_count;
    } else {
        target = 20;
```

```
    }

    unsigned char* acked = (unsigned char*)calloc(msg_count, 1);
// Флаг подтверждения для каждого сообщения.
    size_t acked_count = 0;                                    //
Сколько сообщений уже подтверждено.

    // Сначала один раз отправляем все подготовленные сообщения.
    for (size_t i = 0; i < msg_count; ++i) {
        if (acked[i]) continue;

        sendto(s, msgs[i].datagram, msgs[i].datagram_len,
               0, (sockaddr*)&serv_addr, sizeof(serv_addr));
    }

    // Дальше работаем по схеме:
    // ждём ответ 100 мс, если ответа нет — повторно отправляем
всё неподтверждённое.
    while (acked_count < target) {
        fd_set rfds;
        FD_ZERO(&rfds);
        FD_SET(s, &rfds);

        timeval tv;
        tv.tv_sec = 0;
        tv.tv_usec = 100000; // 100 мс

        int rc = select(s + 1, &rfds, NULL, NULL, &tv);

        if (rc == 0) {
            // Таймаут означает, что ответ от сервера не пришёл.
            // Повторяем отправку всех сообщений, по которым ещё
нет подтверждения.
            for (size_t i = 0; i < msg_count; ++i) {
                if (acked[i]) continue;

                sendto(s, msgs[i].datagram, msgs[i].datagram_len,
                       0,                  (sockaddr*)&serv_addr,
sizeof(serv_addr));
            }
        }

        if (rc > 0 && FD_ISSET(s, &rfds)) {
            unsigned char recvbuf[65535]; // Сюда приходит ответ
сервера.
            sockaddr_in from{};
            socklen_t from_len = sizeof(from);
            ssize_t r = recvfrom(s, recvbuf, sizeof(recvbuf), 0,
(sockaddr*)&from, &from_len);

            if (r > 0 && (r % 4 == 0)) {
                // Ответ сервера — это набор msg_id, по 4 байта на
каждый.
```

```
                for (ssize_t off = 0; off + 4 <= r; off += 4) {
                    uint32_t net_idx;
                    memcpy(&net_idx, recvbuf + off, 4);
                    uint32_t idx = ntohl(net_idx);

                    // Если такой id ещё не был отмечен, считаем
сообщение подтверждённым.
                    if (idx < msg_count && !acked[idx]) {
                        acked[idx] = 1;
                        acked_count++;
                    }
                }
            }
        }
    }

    free(acked);
    for (size_t i = 0; i < msg_count; ++i) {
        free(msgs[i].datagram);
    }
    free(msgs);
    free(line);
    fclose(f);
    close(s);
    return 0;
}
```

Приложение Д «udpserver.cpp»

```
#include <WinSock2.h> /* socket, bind, recvfrom, sendto, closesocket */
#include <WS2tcpip.h> /* sockaddr_in, inet_pton */
#pragma comment(lib, "ws2_32.lib")

#include <windows.h>
#include <iostream>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cstdint>
#include <vector>
#include <string>
#include <ctime>

struct Client{
    uint32_t ip;                        // IP-адрес клиента.
    uint16_t port;                      // Порт клиента.
    time_t last_activity;             // Время последней активности клиента.
    std::vector<uint32_t> received_ids;// Список msg_id, уже полученных от этого клиента.
};

bool CheckPort(char* port1, char* port2, long* port1_, long* port2_){
    // Проверяем корректность границ диапазона портов.
    char* end = NULL;
    long port1_int = strtol(port1,&end,10);
    if(*end!='\0'){
        return false;
    }

    end = NULL;
    long port2_int = strtol(port2,&end,10);
    if(*end!='\0'){
        return false;
    }

    if(port1_int<1 || port1_int>65535) return false;
    if(port2_int<1 || port2_int>65535) return false;
    if(port1_int>port2_int) return false;
    *port1_ = port1_int;
    *port2_ = port2_int;
    return true;

}

bool CreateSockets(long port1, long port2, std::vector<SOCKET>& serv_sockets){
    // Создаём UDP-сокет для каждого порта из диапазона и сразу привязываем его.
```

```
    for(long p=port1;p<=port2;p++){
        SOCKET s_serv = socket(AF_INET, SOCK_DGRAM, 0);
        if (s_serv == INVALID_SOCKET){
            printf("socket failed: %d\n", WSAGetLastError());
            return false;
        }

        // Переводим сокет в неблокирующий режим.
        u_long mode = 1;
        int err = ioctlsocket(s_serv, FIONBIO,&mode);
        if (err!=0){
            printf("Ioctlsocket's                    error:%d\n",
WSAGetLastError());
            closesocket(s_serv);
            return false;
        }

        sockaddr_in addr;
        memset(&addr,0,sizeof(addr));

        addr.sin_family =AF_INET;
        addr.sin_addr.s_addr = htonl(INADDR_ANY); // htonl - 32bit
        addr.sin_port = htons((u_short)p);    // htons - 16bit

        err = bind(s_serv,(sockaddr*)&addr,sizeof(addr));
        if (err!=0){
            printf("Bind's error: %d\n",WSAGetLastError());
            closesocket(s_serv);
            while(!serv_sockets.empty()){
                closesocket(serv_sockets.back());
                serv_sockets.pop_back();
            }
            return false;
        }

        serv_sockets.push_back(s_serv);
    }
    return true;
}

Client CreateClient(uint32_t ip, uint16_t port){
    // Формируем новую запись клиента.
    Client new_client;
    new_client.ip = ip;
    new_client.port = port;
    new_client.last_activity=time(NULL);
    return new_client;
}

int main(int argc,char* argv[]){
    if(argc!=3){
        std::cout    <<    "Usage:    udpserver    <first_port>
<last_port>\n";
```

```
        return 1;
    }
    long port1=0;
    long port2=0;
    if(!CheckPort(argv[1],argv[2],&port1,&port2)){
        std::cout << "Incorrect input\n";
        return 1;
    }

    WSADATA wsa;
    int err = WSAStartup(MAKEWORD(2,2),&wsa);
    if (err!=0){
        printf("WSAStartup error\n");
        return 1;
    }

    std::vector<SOCKET> serv_sockets; // Все серверные сокеты
диапазона портов.

    if(!CreateSockets(port1,port2,serv_sockets)){
        WSACleanup();
        return 1;
    }

    std::vector<WSAPOLLFD> socket_events; // Сюда кладём сокеты
для ожидания через WSAPoll.
    socket_events.resize(serv_sockets.size());
    for(size_t i=0;i<serv_sockets.size();++i){
        socket_events[i].fd = serv_sockets[i];
        socket_events[i].events = POLLIN; // Следим только за
готовностью к чтению.
        socket_events[i].revents = 0;
    }

    bool stop_server = false;    // Флаг завершения сервера по
сообщению "stop".
    std::vector<Client> clients;// Список активных клиентов.
    FILE* msg_file = fopen("msg.txt", "a");
    if (msg_file == NULL) {
        while(!serv_sockets.empty()){
            closesocket(serv_sockets.back());
            serv_sockets.pop_back();
        }
        WSACleanup();
        return 1;
    }
    while(!stop_server){
        // Перед каждым новым ожиданием очищаем revents.
        for (size_t i = 0; i < socket_events.size(); ++i) {
            socket_events[i].revents = 0;
        }

        // Одним вызовом ждём события на всех UDP-сокетах сразу.
```

```
        int      ready      =      WSAPoll(socket_events.data(),
(ULONG)socket_events.size(),1000);
        if (ready == SOCKET_ERROR){
            printf("Ready error: %d\n", WSAGetLastError());
            break;
        }

        for(size_t i = 0; i<socket_events.size();++i){
            if(socket_events[i].revents & POLLIN){
                while(true){
                    char buffer[65536];       // Буфер для одной
входящей дейтаграммы.
                    sockaddr_in client_addr;// Адрес клиента, от
которого пришла дейтаграмма.
                    int client_addr_len = sizeof(client_addr);
                    int received = recvfrom(socket_events[i].fd,
buffer,      sizeof(buffer),      0,      (sockaddr*)&client_addr,
&client_addr_len);
                    if (received == SOCKET_ERROR){
                        int recv_err = WSAGetLastError();
                        if (recv_err == WSAEWOULDBLOCK){
                            break;
                        }
                        printf("Recvfrom error: %d\n", recv_err);
                        break;
                    }

                    //   Если   дейтаграмма   меньше   минимального
формата, пропускаем её.
                    if(received<31){
                        continue;
                    }

                    //  Первые  4  байта  —  это  служебный  номер
сообщения.
                    uint32_t msg_id_net = 0;
                    memcpy(&msg_id_net,buffer,4);

                    uint32_t msg_id = ntohl(msg_id_net);
                    uint32_t ip = client_addr.sin_addr.s_addr;
                    uint16_t port = ntohs(client_addr.sin_port);
                    int client_index = -1;

                    // Ищем клиента по паре IP:port.
                    for(size_t j=0;j<clients.size();j++){
                        if(clients[j].ip == ip && clients[j].port
== port){
                            client_index = j;
                            break;
                        }
                    }
                    Client new_client;
                    if(client_index==-1){
```

```
                    new_client = CreateClient(ip,port);
                    clients.push_back(new_client);
                    client_index = (int)clients.size()-1;
                }

                // Обновляем время последней активности
клиента.
                clients[client_index].last_activity =
time(NULL);

                // Проверяем, не был ли этот msg_id уже
получен раньше.
                bool already_received = false;
                for (size_t k = 0; k <
clients[client_index].received_ids.size(); ++k) {
                    if (clients[client_index].received_ids[k]
== msg_id) {
                        already_received = true;
                        break;
                    }
                }

                // Разбираем поля сообщения из бинарного
буфера.
                std::string phone1(buffer+4,12);
                std::string phone2(buffer+16,12);

                unsigned char hh = (unsigned char)buffer[28];
                unsigned char mm = (unsigned char)buffer[29];
                unsigned char ss = (unsigned char)buffer[30];

                char* msgStart = buffer + 31; // Текст
сообщения начинается после фиксированной части.
                int msgLen = received - 31;

                // Последний байт текста — это '\0', поэтому
в std::string его не включаем.
                std::string messageText(msgStart, msgLen -
1);

                if (!already_received) {
                    // Новое сообщение сохраняем в памяти
клиента и в msg.txt.

clients[client_index].received_ids.push_back(msg_id);


                    char ip_str[INET_ADDRSTRLEN];
                    inet_ntop(AF_INET, &client_addr.sin_addr,
ip_str, sizeof(ip_str));

                    fprintf(
                        msg_file,
```

```
                        "%s:%u %s %s %02u:%02u:%02u %s\n",
                        ip_str,
                        (unsigned int)port,
                        phone1.c_str(),
                        phone2.c_str(),
                        (unsigned int)hh,
                        (unsigned int)mm,
                        (unsigned int)ss,
                        messageText.c_str()
                    );


                }

                // Формируем ответ клиенту: в него кладём до
20 последних msg_id,
                // которые сервер уже принял от этого клиента.
                std::vector<char> sendbuf;
                size_t           size_client_ids           =
clients[client_index].received_ids.size();
                size_t count = 0;
                for(size_t   j   =   size_client_ids;j>0   &&
count<20; --j){
                    uint32_t         msg_id_to_send         =
clients[client_index].received_ids[j - 1];
                    uint32_t net_id = htonl(msg_id_to_send);
                    char* p = (char*)&net_id;
                    sendbuf.push_back(p[0]);
                    sendbuf.push_back(p[1]);
                    sendbuf.push_back(p[2]);
                    sendbuf.push_back(p[3]);
                    ++count;
                }
                int sent = sendto(
                socket_events[i].fd,
                sendbuf.data(),
                (int)sendbuf.size(),
                0,
                (sockaddr*)&client_addr,
                client_addr_len
                );

                if (sent == SOCKET_ERROR) {
                    printf("Sendto       error:       %d\n",
WSAGetLastError());
                }

                // Сообщение "stop" не останавливает сервер
мгновенно посреди цикла,
                // а только ставит флаг завершения.
                if (messageText == "stop"){
                    stop_server=true;
                }
```

```
                }

            }

        }

        // Периодически удаляем клиентов, от которых давно не было
активности.
        time_t now = time(NULL);
        size_t j = 0;

        while (j < clients.size()) {
            if (clients[j].last_activity + 30 < now) {
                clients.erase(clients.begin() + j);
            } else {
                ++j;
            }
        }

    }

    fclose(msg_file);

    while(!serv_sockets.empty()){
        closesocket(serv_sockets.back());
        serv_sockets.pop_back();
    }

    WSACleanup();
    return 0;
}
```